

# The Interactive MRI $k$ -Space Simulator

A Complete Concept-First Guide

From “What is a frequency?” to a Finished Desktop Application

Theory in plain English · Mathematics explained line-by-line · Implementation as checklists

Companion guide for the 6-week roadmap  
(BUET CSE 219 / CSE 220)

August 12, 2026

## Abstract

This document teaches you *everything* you need to build the Interactive MRI  $k$ -Space Reconstruction Simulator — and, more importantly, to *understand* it deeply enough to answer any question about it. Every chapter follows the same rhythm: first the **concept in plain English** (with analogies), then the **mathematics explained gently**, then a **diagram** so you can *see* the idea, then a **step-by-step implementation checklist** (deliberately without code — the bullets tell you *what* to build, so the code stays yours), and finally **tests and self-check questions**. Nothing in this project is magic. It is all matrix arithmetic, applied patiently.

# Contents

|          |                                                                                                 |           |
|----------|-------------------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>The Big Picture: What Are We Actually Building?</b>                                          | <b>3</b>  |
| 1.1      | The one-paragraph version of the whole project . . . . .                                        | 3         |
| 1.2      | The five ideas the entire application rests on . . . . .                                        | 3         |
| 1.3      | The data-flow pipeline, visually . . . . .                                                      | 4         |
| 1.4      | The modules you will create, and what each one owns . . . . .                                   | 4         |
| <b>2</b> | <b>Milestone 1 — Images, Waves, and the Fourier Transform</b>                                   | <b>6</b>  |
| 2.1      | Step 1: An image is just a matrix . . . . .                                                     | 6         |
| 2.2      | Step 2: What is a “spatial frequency”? . . . . .                                                | 6         |
| 2.3      | Step 3: The Fourier transform, gently . . . . .                                                 | 7         |
| 2.3.1    | The continuous version (the “textbook” formula) . . . . .                                       | 7         |
| 2.3.2    | The discrete version (the one your computer runs) . . . . .                                     | 8         |
| 2.3.3    | $k$ -space values are complex numbers — and that is important . . . . .                         | 8         |
| 2.4      | Step 4: <code>fftshift</code> — putting the recipe’s center in the middle . . . . .             | 9         |
| 2.5      | Step 5: Why we display $\log(1 +  K )$ . . . . .                                                | 9         |
| 2.6      | Step 6: The anatomy of $k$ -space . . . . .                                                     | 9         |
| 2.7      | Step 7: Parseval’s theorem — your built-in correctness test . . . . .                           | 9         |
| 2.8      | Implementation guide . . . . .                                                                  | 10        |
| <b>3</b> | <b>Milestone 2 — Sampling, Undersampling, and Aliasing</b>                                      | <b>12</b> |
| 3.1      | Why scanning takes time — and why we cheat . . . . .                                            | 12        |
| 3.2      | The Nyquist rule in plain English . . . . .                                                     | 12        |
| 3.3      | Sampling masks: undersampling as multiplication . . . . .                                       | 13        |
| 3.4      | The Point Spread Function: predicting the artifact before you see it . . . . .                  | 14        |
| 3.5      | Implementation guide . . . . .                                                                  | 14        |
| <b>4</b> | <b>Milestone 3 — Gibbs Ringing, Apodization, and Noise</b>                                      | <b>17</b> |
| 4.1      | The problem: sharp scissors leave ripples . . . . .                                             | 17        |
| 4.2      | The fix: apodization windows . . . . .                                                          | 18        |
| 4.3      | Thermal noise: why real scans are grainy . . . . .                                              | 19        |
| 4.4      | Implementation guide . . . . .                                                                  | 19        |
| <b>5</b> | <b>Milestone 4 — Radial &amp; Spiral Trajectories, Density Compensation, and Grid-<br/>ding</b> | <b>22</b> |
| 5.1      | Why scanners abandon straight lines . . . . .                                                   | 22        |
| 5.2      | Drawing the paths: just trigonometry . . . . .                                                  | 23        |
| 5.3      | The density problem: spokes crowd the center . . . . .                                          | 23        |
| 5.4      | Gridding: getting scattered points back onto the FFT’s grid . . . . .                           | 23        |
| 5.5      | What the artifacts look like now . . . . .                                                      | 25        |
| 5.6      | Implementation guide . . . . .                                                                  | 25        |
| <b>6</b> | <b>Milestone 5 — Measuring Image Quality: MSE, PSNR, and SSIM</b>                               | <b>27</b> |
| 6.1      | From “it looks worse” to numbers . . . . .                                                      | 27        |
| 6.2      | MSE: the average squared mistake . . . . .                                                      | 27        |
| 6.3      | PSNR: MSE on a decibel scale . . . . .                                                          | 27        |
| 6.4      | SSIM: asking what a human would say . . . . .                                                   | 27        |
| 6.5      | The error heatmap: seeing <i>where</i> it went wrong . . . . .                                  | 28        |
| 6.6      | Implementation guide . . . . .                                                                  | 28        |

|           |                                                                            |           |
|-----------|----------------------------------------------------------------------------|-----------|
| <b>7</b>  | <b>Milestone 6 — The Interactive GUI and Application Architecture</b>      | <b>30</b> |
| 7.1       | From engine to instrument . . . . .                                        | 30        |
| 7.2       | The layout . . . . .                                                       | 30        |
| 7.3       | The one problem that ruins naive GUIs: freezing . . . . .                  | 31        |
| 7.4       | The architecture that keeps six weeks of code sane . . . . .               | 31        |
| 7.5       | Implementation guide . . . . .                                             | 32        |
| <b>8</b>  | <b>The Build Plan — Order of Work, Edge Cases, and Polish</b>              | <b>34</b> |
| 8.1       | Why this exact order . . . . .                                             | 34        |
| 8.2       | Edge cases you must handle (Week 6 checklist) . . . . .                    | 34        |
| 8.3       | When you get stuck: the debugging ladder . . . . .                         | 35        |
| <b>9</b>  | <b>Reference: Glossary and Examiner’s Question Bank</b>                    | <b>36</b> |
| 9.1       | Glossary — every intimidating term, in one sentence each . . . . .         | 36        |
| 9.2       | The examiner’s question bank . . . . .                                     | 37        |
| <b>10</b> | <b>Team Onboarding — Environment Setup and the Project Map</b>             | <b>40</b> |
| 10.1      | What must already be on your machine . . . . .                             | 40        |
| 10.2      | Setup, step by step . . . . .                                              | 40        |
| 10.3      | The project map . . . . .                                                  | 41        |
| <b>11</b> | <b>PySide6 Field Guide — Just Enough Qt, in Plain English</b>              | <b>43</b> |
| 11.1      | Idea 1: the event loop — your program becomes a waiter . . . . .           | 43        |
| 11.2      | Idea 2: widgets and layouts — Lego bricks that resize themselves . . . . . | 43        |
| 11.3      | Idea 3: signals and slots — widgets announce, they never command . . . . . | 44        |
| 11.4      | Embedding matplotlib: the canvas is just another widget . . . . .          | 45        |
| 11.5      | Keeping it responsive: the two-line summary of Chapter 7 . . . . .         | 45        |
| 11.6      | Making it look good: the two-step styling recipe . . . . .                 | 45        |
| 11.7      | The five classic beginner traps . . . . .                                  | 46        |

# The Big Picture: What Are We Actually Building?

## 1.1 The one-paragraph version of the whole project

An MRI scanner never takes a photograph. It measures *radio signals* that correspond to the *frequencies* hidden inside the patient's body slice, and a computer then *reconstructs* a picture from those frequencies using the mathematics of the Fourier transform. Your project builds a desktop application that simulates this entire process on ordinary images: it converts an image into its frequency representation (called *k-space*), lets the user damage or modify that frequency data in all the ways a real scanner does (skipping lines to scan faster, adding electrical noise, sampling along circles and spirals instead of straight rows), reconstructs the image from the damaged data, and *measures* exactly how much image quality was lost. Sliders and dropdowns make every effect interactive and instant.

### Analogy

Think of the scanner as a musician with perfect pitch sitting in a concert hall, eyes closed. The musician cannot *see* the orchestra (the image), but hears every individual note being played (the frequencies). If you write down the full list of notes and their volumes, you can later reconstruct exactly what the orchestra sounded like. *k-space* is that written list of notes. The Fourier transform is the perfect-pitch skill that converts between “sound of the whole orchestra” and “list of individual notes,” in both directions, without losing anything.

Now imagine the musician gets lazy and only writes down every second note (undersampling), or the hall has an air conditioner humming in the background (noise), or the musician scribbles the notes in a spiral around the page instead of neat rows (non-Cartesian sampling). Each of these corrupts the reconstruction in a *specific, predictable* way — and your application demonstrates every one of them.

## 1.2 The five ideas the entire application rests on

Everything in the roadmap — every module, slider, and formula — is an application of just five ideas. Read them now, and re-read them after every chapter; you will watch them become obvious.

1. **An image is a matrix of numbers.** A grayscale image of size  $256 \times 256$  is nothing more than a table of 65,536 brightness values between 0.0 (black) and 1.0 (white).
2. **Any image can be rewritten as a sum of waves.** Smooth, slow waves capture the overall shapes and contrast; fast, tight waves capture edges and fine texture. The recipe of “which waves, how strong, and how shifted” is *k-space*. (Chapter 2)
3. **Removing waves from the recipe damages the picture in predictable ways.** Remove rows of the recipe → ghost copies of the image appear (aliasing). Chop off the outer recipe abruptly → rippling rings appear near edges (Gibbs ringing). (Chapters 3 and 4)
4. **The reconstruction machinery only accepts recipes written on a neat square**

**grid.** If the scanner collected the recipe along spokes or spirals, we must first *re-grid* it — carefully, compensating for the fact that spokes crowd together in the middle. (Chapter 5)

5. **Damage can be measured with three numbers.** MSE, PSNR, and SSIM turn “this image looks worse” into quantities you can plot against a slider. (Chapter 6)

### 1.3 The data-flow pipeline, visually

Every interaction in the finished application — every slider drag — pushes an image through this pipeline exactly once. Keep this figure in your head; each chapter of this guide simply zooms into one of these boxes.

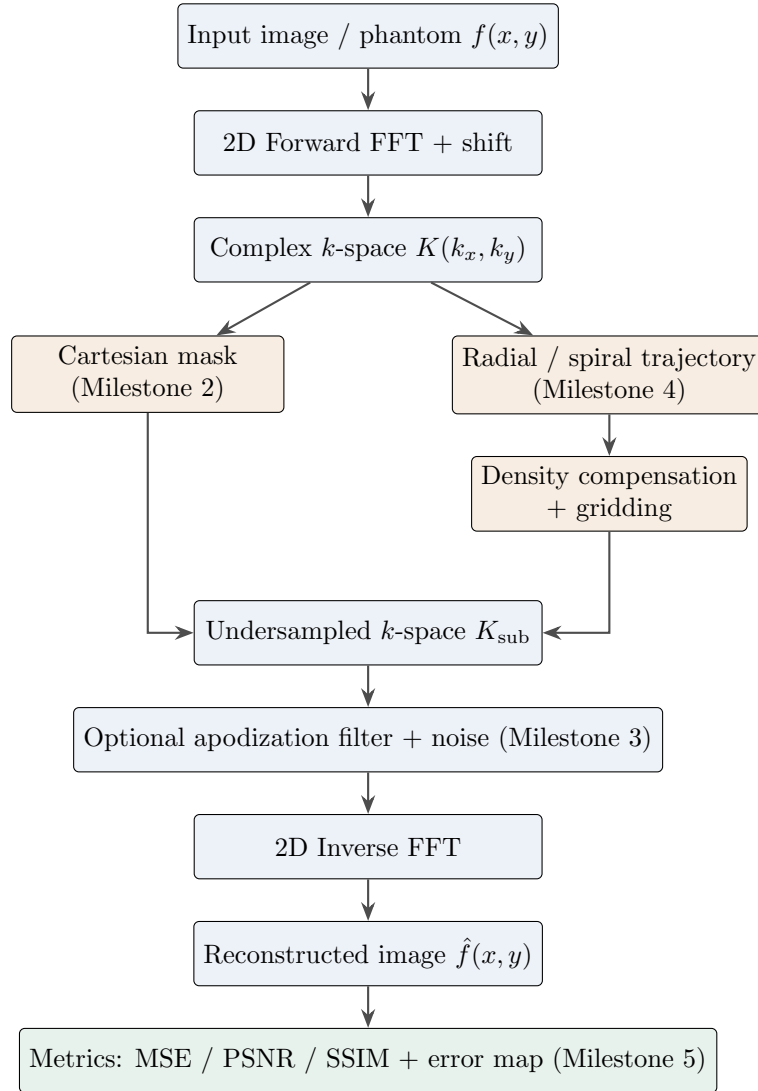


Figure 1.1: The complete data-flow pipeline of the simulator. The GUI (Milestone 6) is simply a set of sliders that re-runs this pipeline and four panels that display its intermediate results.

### 1.4 The modules you will create, and what each one owns

The application is split into small files, each owning one box of Figure 1.1. This separation is what makes a six-week project survivable: each file can be written and tested *alone*, in a terminal, before any GUI exists.

| File                               | Owns                                              | Guide chapter     |
|------------------------------------|---------------------------------------------------|-------------------|
| <code>kspace_core.py</code>        | FFT, IFFT, shifting, log display, phantom loading | <a href="#">2</a> |
| <code>cartesian_sampling.py</code> | Binary masks, acceleration factor, PSF            | <a href="#">3</a> |
| <code>filters.py</code>            | Hamming/Gaussian windows, noise injection         | <a href="#">4</a> |
| <code>trajectories.py</code>       | Radial and spiral coordinates, density weights    | <a href="#">5</a> |
| <code>gridding.py</code>           | Interpolation of scattered points onto the grid   | <a href="#">5</a> |
| <code>metrics.py</code>            | MSE, PSNR, SSIM, error heatmaps                   | <a href="#">6</a> |
| <code>app.py</code>                | Windows, panels, sliders, worker threads          | <a href="#">7</a> |

### Key Takeaways

- The scanner records **frequencies**, not pictures; reconstruction is mathematics, and that mathematics is the 2D Fourier transform.
- Your app is one pipeline (Figure [1.1](#)) run over and over with different settings.
- Every “advanced” term in the roadmap (PSF, apodization, density compensation, gridding) is a small idea wearing an intimidating name. Each gets its own gentle chapter here.

# Milestone 1 — Images, Waves, and the Fourier Transform

This is the foundation chapter. If you understand it well, every later chapter becomes a variation on it. Take your time here.

## 2.1 Step 1: An image is just a matrix

### The Concept in Plain English

A grayscale digital image is a rectangular table of numbers. Each cell of the table is one **pixel**, and the number inside it says how bright that pixel is. In this project we standardize the numbers to lie between 0.0 (pure black) and 1.0 (pure white). A  $256 \times 256$  image is a matrix with 256 rows and 256 columns — 65,536 numbers total. That is all an image is. Everything we do to images in this project is therefore *matrix arithmetic*: adding matrices, multiplying them element by element, and transforming them.

Mathematically we write the image as  $f[m, n]$ , where  $m$  is the row index and  $n$  is the column index, each running from 0 to  $N - 1$  for an  $N \times N$  image. Two practical notes that will save you debugging later:

- **Color images must be collapsed to one channel.** A color pixel has three numbers (red, green, blue). MRI images are single-channel, so we convert using the standard luminance formula, which weights the channels the way the human eye does:

$$Y = 0.299 R + 0.587 G + 0.114 B. \quad (2.1)$$

- **Sizes should be powers of two.** The Fast Fourier Transform is at its fastest when the matrix dimensions are  $2^k$  (e.g. 256 or 512). We therefore resize (or zero-pad) every input to  $256 \times 256$ .

## 2.2 Step 2: What is a “spatial frequency”?

### The Concept in Plain English

You know frequency from sound: a high-pitched note wiggles quickly in *time*, a low-pitched note wiggles slowly. A **spatial** frequency is the same idea applied to *space* instead of time. Imagine painting smooth stripes across a canvas, fading from black to white and back like a wave. If the stripes are wide and lazy, that is a *low* spatial frequency. If the stripes are thin and packed tightly, that is a *high* spatial frequency. Such a striped pattern is called a **grating**, and it is the 2D equivalent of a single pure musical note.

A grating has three properties: (1) **frequency** — how many stripe cycles fit across the image; (2) **orientation** — stripes can run vertically, horizontally, or at any angle; and (3) **phase** — where the bright stripe sits (shifting the whole pattern sideways).

## Analogy

The central claim of Fourier analysis, in one sentence: *any* image — a brain scan, a photograph of your cat, anything — can be built by stacking up enough gratings of different frequencies, orientations, phases, and strengths, the same way any chord can be built from single notes. The Fourier transform is the machine that takes the finished image and hands you back the complete recipe: the exact list of gratings and their strengths.

To make this concrete, here is the idea in one dimension. Watch how adding just a few pure sine waves already starts building a sharp-edged square wave — edges (sharp features) require *many high frequencies*:

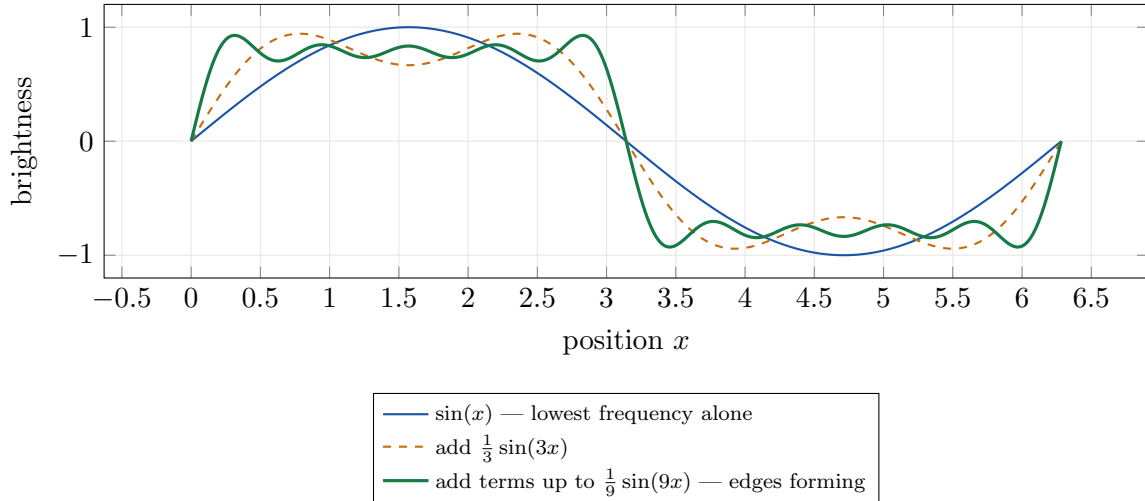


Figure 2.1: Summing sine waves builds sharper and sharper shapes. Low frequencies sketch the rough outline; high frequencies carve the sharp edges. The same is true in 2D for images.

## Why This Matters

This single figure explains two later phenomena before you even meet them:

- If we *delete high frequencies* (Chapters 3 and 4), edges cannot be formed sharply anymore → the image blurs.
- If we *cut the series off abruptly*, the sum overshoots and ripples near edges — you can already see small wiggles in the green curve. That rippling is **Gibbs ringing** (Chapter 4).

## 2.3 Step 3: The Fourier transform, gently

### 2.3.1 The continuous version (the “textbook” formula)

For a continuous image  $f(x, y)$ , the 2D Fourier transform and its inverse are:

$$K(k_x, k_y) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(x, y) e^{-i2\pi(k_x x + k_y y)} dx dy, \quad (2.2)$$

$$f(x, y) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} K(k_x, k_y) e^{+i2\pi(k_x x + k_y y)} dk_x dk_y. \quad (2.3)$$

Do not let the integrals intimidate you. Read Equation (2.2) as a recipe-extraction procedure:

- The term  $e^{-i2\pi(k_x x + k_y y)}$  is a grating — a single wavy pattern with  $k_x$  cycles per unit distance horizontally and  $k_y$  vertically. (Euler’s formula,  $e^{i\theta} = \cos \theta + i \sin \theta$ , says a complex exponential is just a cosine and a sine travelling together.)
- Multiplying the image by that grating and integrating (summing over the whole image) asks one question: “*How much of this exact grating is present in this image?*” If the image contains a lot of that pattern, the products reinforce and the integral is large. If not, positive and negative contributions cancel and the integral is nearly zero.
- $K(k_x, k_y)$  therefore stores, at position  $(k_x, k_y)$ , the answer for the grating of that frequency and orientation. The whole function  $K$  is the complete recipe — **this is  $k$ -space**.

Equation (2.3) is the reverse: take every grating, scale it by its recipe entry, add them all up, and the image reappears. The only difference between the two formulas is the sign in the exponent — decomposition vs. reassembly.

### 2.3.2 The discrete version (the one your computer runs)

Computers hold matrices, not continuous functions, so we use the Discrete Fourier Transform (DFT). For an  $N \times M$  image:

$$K[u, v] = \sum_{m=0}^{N-1} \sum_{n=0}^{M-1} f[m, n] e^{-i2\pi(\frac{um}{N} + \frac{vn}{M})}, \quad (2.4)$$

$$f[m, n] = \frac{1}{NM} \sum_{u=0}^{N-1} \sum_{v=0}^{M-1} K[u, v] e^{+i2\pi(\frac{um}{N} + \frac{vn}{M})}. \quad (2.5)$$

Integrals became sums; everything else is identical. The indices  $u$  and  $v$  count *how many full cycles* the grating completes across the image:  $u = 0$  is “no variation at all” (a flat sheet),  $u = 1$  is one full wave across the image,  $u = 10$  is ten tight waves, and so on.

The **FFT** (Fast Fourier Transform) is not a different transform — it is a clever algorithm that computes exactly Equation (2.4), but in  $\mathcal{O}(N^2 \log N)$  operations instead of the naive  $\mathcal{O}(N^4)$ . That speed difference is what makes real-time sliders possible, and it is why we prefer power-of-two image sizes.

### 2.3.3 $k$ -space values are complex numbers — and that is important

Each entry  $K[u, v]$  is a **complex number**  $a + bi$ . Why? Because each grating needs *two* facts recorded about it, and a complex number stores exactly two:

$$\textbf{Magnitude} \quad |K[u, v]| = \sqrt{a^2 + b^2} \quad \text{— } \textit{how strong is this grating?} \quad (2.6)$$

$$\textbf{Phase} \quad \angle K[u, v] = \text{atan2}(b, a) \quad \text{— } \textit{how shifted is it sideways?} \quad (2.7)$$

#### Why This Matters

A classic experiment (worth reproducing in your app!): reconstruct an image using only the magnitudes (set all phases to zero) — you get an unrecognizable blob. Reconstruct using only the phases (set all magnitudes to one) — the edges and outlines of the original image are clearly visible. **Phase carries the structure; magnitude carries the contrast strength.** This is why we must keep  $k$ -space complex throughout the pipeline and only take magnitudes at the very final display step.

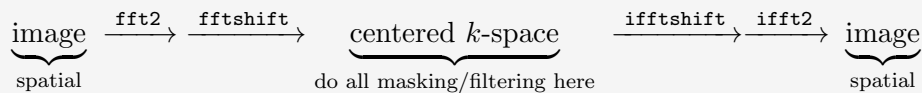
## 2.4 Step 4: fftshift — putting the recipe’s center in the middle

The raw FFT output stores the zero-frequency entry (called the **DC component** — the flat “average brightness” grating) in the *top-left corner* of the matrix, with negative frequencies wrapped around to the far side. That is a storage convention, nothing deeper. For humans (and for masks that assume “center = low frequency”), we want the DC term in the *middle* of the matrix, with frequency increasing outward in every direction.

`fftshift` performs exactly this rearrangement: it swaps the quadrants of the matrix so that the DC entry lands at the center. `ifftshift` undoes it. The golden rule of the whole project:

### Key Takeaways

**The sandwich rule.** Every trip into  $k$ -space and back follows the same symmetric pattern:



If you ever see a reconstruction that looks like a checkerboard was multiplied into your image, you forgot one of the shifts. This is the single most common bug in the entire project.

## 2.5 Step 5: Why we display $\log(1 + |K|)$

The DC component and its low-frequency neighbors are *astronomically* larger than everything else — often a million times larger — because most images are dominated by smooth regions. If you display  $|K|$  directly, you see one white pixel in the center and blackness everywhere else. The fix is logarithmic compression:

$$V[u, v] = \log(1 + |K[u, v]|). \quad (2.8)$$

The “1+” guarantees the argument is never zero (avoiding  $\log 0 = -\infty$ ), and the logarithm squeezes the enormous dynamic range into something a screen can show. This is *display only* — never feed the log version back into the reconstruction.

## 2.6 Step 6: The anatomy of $k$ -space

This geography is the key to *predicting* what any manipulation will do: delete the periphery  $\rightarrow$  blurry but recognizable image; delete the center  $\rightarrow$  ghostly outline showing only edges; a mask’s shape in  $k$ -space determines the artifact’s shape in the image.

## 2.7 Step 7: Parseval’s theorem — your built-in correctness test

Parseval’s theorem says the total “energy” of an image is preserved by the Fourier transform (up to a known scaling constant):

$$\sum_{m,n} |f[m, n]|^2 = \frac{1}{NM} \sum_{u,v} |K[u, v]|^2. \quad (2.9)$$

In plain English: rewriting the image as a recipe of gratings neither creates nor destroys anything — it is a lossless change of viewpoint, like weighing ingredients before and after mixing. Practically, Equation (2.9) gives you a *free automated test*: compute both sides; if they differ by more than floating-point dust ( $\sim 10^{-10}$  relative), your FFT plumbing has a bug.

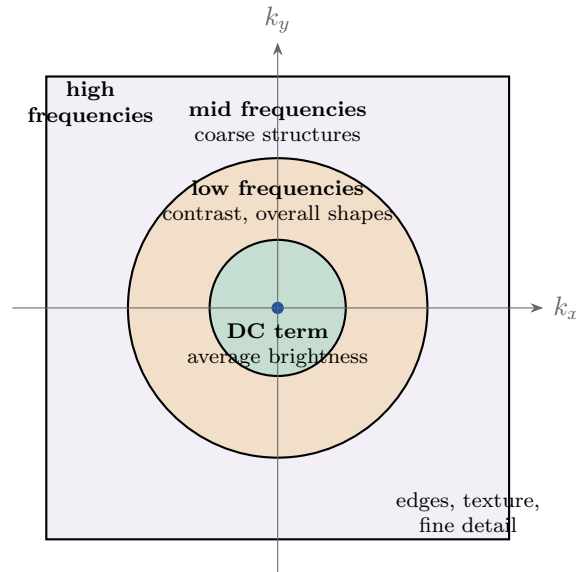


Figure 2.2: The anatomy of centered  $k$ -space. Distance from the center equals spatial frequency. The tiny central region carries almost all the image *energy* (contrast); the vast periphery carries the *detail*.

## 2.8 Implementation guide

### What to Build — Step-by-Step (No Code, Just Instructions)

All of this lives in `kspace_core.py`. Work in a plain script or Jupyter notebook with `matplotlib` — absolutely no GUI code yet.

#### A. The image loader

- Write a loader that accepts a file path and returns a clean matrix: open the image, convert to grayscale using the luminance formula, convert the data type to 64-bit floating point.
- Resize the result to  $256 \times 256$ . Decide and document your policy for non-square inputs (simplest: resize both dimensions; better: pad with zeros to a square first so shapes are not distorted).
- Normalize: divide by the maximum value so all pixels lie in  $[0, 1]$ .
- Add a second loader that produces the **Shepp–Logan phantom** (a standard synthetic “fake head” image available in `scikit-image`) so you always have a canonical test image with known properties.

#### B. The forward transform

- Write a function that takes the image matrix and returns centered  $k$ -space: apply the 2D FFT, then apply the quadrant-swapping shift so the DC term sits at the center. Keep the result *complex* — do not take magnitudes here.
- Write two small helper functions on top of it: one returning the **magnitude spectrum** (with an optional log-compression flag for display), and one returning the **phase spectrum** using the two-argument arctangent of the imaginary and real parts.

#### C. The inverse transform

- Write a function that takes any centered  $k$ -space matrix and returns the reconstructed image: undo the shift first, then apply the 2D inverse FFT, then take the magnitude

(absolute value) of the complex result.

- Make it accept *any*  $k$ -space matrix as input — later milestones will feed it masked, filtered, noisy, and re-gridded versions.

#### D. The frequency-probing experiment (builds intuition you will need)

- Write a function that keeps only a central disc of  $k$ -space (radius  $r$  pixels, everything outside set to zero) and reconstructs — a **low-pass filter**.
- Write its twin that deletes the central disc and keeps the periphery — a **high-pass filter**.
- Reconstruct the phantom with each, at several radii, and save the side-by-side plots. You should personally observe: low-pass = blurry but recognizable; high-pass = dark image showing only edges and outlines.

#### How to Test It Before Moving On

- **Round-trip test:** image  $\rightarrow$  forward  $\rightarrow$  inverse with no modification. The maximum absolute pixel difference from the original must be below  $10^{-12}$ . If not, check your shift sandwich.
- **Parseval test:** verify Equation (2.9) holds to within  $10^{-10}$  relative error.
- **Visual test:** display the log-magnitude spectrum of the phantom. You should see a bright star-like glow exactly at the center of the matrix, fading outward, with faint spokes. If the glow is at the four corners instead, you forgot the centering shift.
- **DC sanity test:** the magnitude at the exact center pixel should equal the sum of all image pixels (that is literally what the  $u=v=0$  term of Equation (2.4) computes). Check it numerically.

#### Self-Check: Can You Answer These?

1. In one sentence: what does the value stored at position  $(u, v)$  of  $k$ -space tell you?
2. Why must  $k$ -space entries be complex numbers? What do magnitude and phase each encode?
3. What exactly does `fftshift` move, and why do we want it moved?
4. Why is the center of  $k$ -space so much brighter than the rest, and how do we display it anyway?
5. If you delete the outer ring of  $k$ -space, what happens to the image — and *why*, in terms of Figure 2.1?
6. State Parseval's theorem in plain English. How would you use it as a unit test?
7. Which reconstruction looks more like the original image: magnitude-only or phase-only? What does that tell you about where structure lives?

# Milestone 2 — Sampling, Undersampling, and Aliasing

## 3.1 Why scanning takes time — and why we cheat

### The Concept in Plain English

A real MRI scanner cannot capture all of  $k$ -space at once. It acquires it **one horizontal line at a time**: play a magnetic gradient, record one row of frequency samples, wait for the physics to reset, record the next row. Each row is called a **phase-encoding line**, and a full 256-line acquisition can take minutes. Patients move, breathe, and get uncomfortable — so there is enormous clinical pressure to *skip lines* and scan faster.

Skipping lines is called **undersampling**. If we acquire only every second line we scan twice as fast; the speed-up is called the **acceleration factor**:

$$R = \frac{\text{lines a full scan would need}}{\text{lines actually acquired}} = \frac{N_{\text{total}}}{N_{\text{sampled}}}.$$

But mathematics charges a price for skipped information, and the price is a strange, beautiful artifact called **aliasing**. Understanding exactly why it happens — and why it looks the way it looks — is the point of this milestone.

## 3.2 The Nyquist rule in plain English

### Analogy

You have seen aliasing before: in old westerns, wagon wheels sometimes appear to spin *backwards*. The camera samples the world 24 times per second; if the wheel spokes rotate faster than the camera can keep up with, the brain is fed misleading snapshots and perceives a slower (or reversed) motion. A fast thing, sampled too rarely, *masquerades as a slow thing*. “Alias” literally means “false identity.”

The precise statement is the **Nyquist–Shannon criterion**: to capture a signal faithfully, you must take samples at least twice as often as the highest frequency present in it. In MRI the “samples” are  $k$ -space lines and the rule takes a spatial form: the spacing between acquired  $k$ -space lines,  $\Delta k$ , determines the size of the region the image can faithfully represent, called the **field of view**:

$$\text{FOV} = \frac{1}{\Delta k}. \quad (3.1)$$

Read it as a see-saw: *finely spaced*  $k$ -space lines  $\rightarrow$  large FOV (room for the whole head); *coarsely spaced* lines (because we skipped some)  $\rightarrow$  the FOV *shrinks*:

$$\text{FOV}_{\text{eff}} = \frac{\text{FOV}}{R}. \quad (3.2)$$

And here is the punchline: the object does *not* shrink with it. An object larger than the effective FOV has nowhere to go — so the mathematics wraps it around, like a word-processor wrapping

a too-long line. Copies of the anatomy fold back on top of itself. Radiologists call this **wrap-around** or **ghosting**.

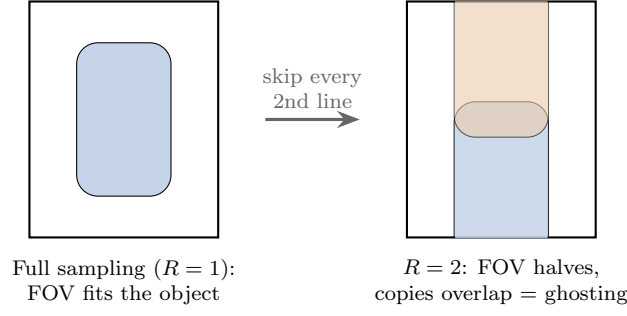


Figure 3.1: Uniform undersampling by  $R=2$  halves the effective field of view along the skipped direction. The anatomy no longer fits, so a shifted duplicate wraps around and overlaps the original.

### 3.3 Sampling masks: undersampling as multiplication

How do we *simulate* skipping lines when we already have the full  $k$ -space? With a **binary mask**: a matrix  $S[u, v]$  the same size as  $k$ -space containing 1 where a sample was “acquired” and 0 where it was “skipped.” Undersampling is then a single element-wise multiplication:

$$K_{\text{sub}}[u, v] = K[u, v] \cdot S[u, v]. \quad (3.3)$$

This is the entire trick of Milestone 2 — everything else is choosing interesting patterns of ones and zeros. Your app implements four:

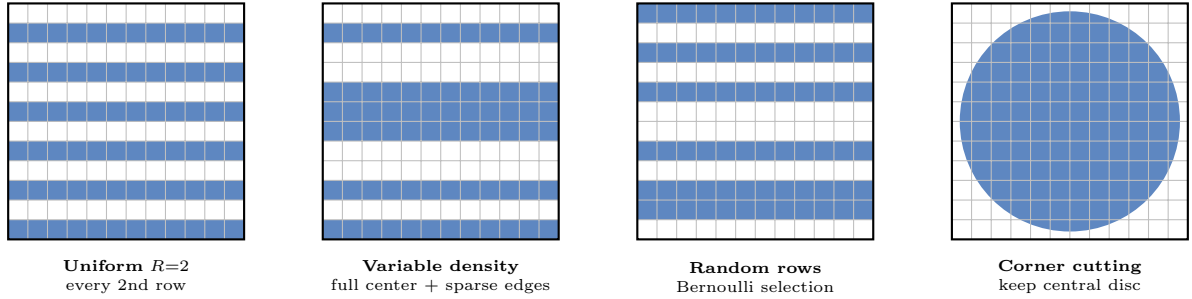


Figure 3.2: The four Cartesian sampling masks (blue = acquired = 1, white = skipped = 0). Each produces a visually distinct artifact in the reconstruction.

1. **Uniform subsampling.** Keep every  $R$ -th row:

$$S_{\text{uniform}}[u, v] = \begin{cases} 1 & \text{if } v \bmod R = 0 \\ 0 & \text{otherwise.} \end{cases} \quad (3.4)$$

Produces the classic, *coherent* ghost copies of Figure 3.1 — regular skipping creates regular ghosts.

2. **Variable-density sampling (VDS).** Fully sample a central band (the low frequencies, which hold nearly all the energy — recall Figure 2.2) and sample the periphery sparsely, keeping each outer row with some probability. Preserves overall contrast far better at the same  $R$ ; the artifact becomes milder and less structured.

3. **Random subsampling.** Keep each row with probability  $1/R$ , decided by a coin flip. Regularity is what makes ghosts crisp, so randomness smears the artifact into incoherent, noise-like grain. (This idea — random undersampling turns aliasing into “noise” that clever algorithms can remove — is the foundation of *compressed sensing* MRI.)
4. **Corner cutting / circular mask.** Keep only  $u^2 + v^2 \leq r_{\max}^2$  (measured from the center). This is not row-skipping but frequency truncation: a pure low-pass filter. It blurs the image and, because the cutoff is abrupt, introduces the Gibbs ringing that Milestone 3 will fix.

### 3.4 The Point Spread Function: predicting the artifact before you see it

#### The Concept in Plain English

Here is the most elegant idea in the milestone. Instead of asking “what does this mask do to *my image*?”, ask a purer question: “what does this mask do to a *single bright dot*?” The answer is called the **Point Spread Function (PSF)**. A perfect imaging system leaves a dot as a dot. A mask smears the dot into a pattern — and *that pattern gets stamped onto every point of any image* you reconstruct through the mask.

The mathematics behind this is the **convolution theorem**, the most important theorem of the course: *multiplication in  $k$ -space equals convolution in image space.*

$$K_{\text{sub}} = K \cdot S \iff \hat{f} = f ** \text{PSF}, \quad \text{PSF} = \mathcal{F}_2^{-1}\{S\}, \quad (3.5)$$

where  $**$  denotes 2D convolution — “stamp a copy of the PSF at every pixel of  $f$ , scaled by that pixel’s brightness, and add them all up.”

This turns artifact analysis into a lookup table:

| Mask          | PSF looks like                        | So the artifact is         |
|---------------|---------------------------------------|----------------------------|
| Uniform $R$   | one central spike + $R-1$ side spikes | $R-1$ shifted ghost copies |
| Random rows   | central spike + scattered low ripple  | noise-like grain           |
| Corner cut    | a blob with concentric rings (a jinc) | blur + ringing at edges    |
| Full sampling | a single perfect spike                | no artifact at all         |

The side spikes of the uniform mask’s PSF *are* the ghosts: convolving the image with “a spike here plus a spike shifted by  $\text{FOV}/R$ ” literally means “the image plus a copy of itself shifted by  $\text{FOV}/R$ .” The convolution theorem does not just describe aliasing — it *explains* it.

### 3.5 Implementation guide

#### What to Build — Step-by-Step (No Code, Just Instructions)

This milestone lives in `cartesian_sampling.py` (mask generation) and `psf_analyzer.py` (PSF study). Still no GUI — work with saved plots.

##### A. The mask generator

- Create a generator class or module whose every function returns a matrix of zeros and ones with the same shape as your  $k$ -space ( $256 \times 256$ ).
- **Uniform:** start from an all-zeros matrix; set entire rows to one wherever the row index is divisible by  $R$ . Accept  $R$  as a parameter.

- **Variable density:** set a central horizontal band (a parameter, e.g. the middle 16–32 rows) fully to one; for every other row, draw a random number and keep the row if it falls below a keep-probability that you derive from the requested  $R$ .
- **Random:** keep each row independently with probability  $1/R$ . Accept an optional random seed parameter so experiments are reproducible.
- **Corner cutting:** build two coordinate matrices holding each entry's horizontal and vertical distance from the matrix center; set to one every entry whose radius  $\sqrt{u^2 + v^2}$  is below the cutoff parameter.
- Add a small reporting helper that takes any mask and returns its *actual* acceleration factor: total entries divided by the number of ones. (For VDS and random masks the achieved  $R$  differs from the requested one — display both.)

## B. Wiring masks into the pipeline

- Multiply the centered  $k$ -space (from Milestone 1) element-wise with the mask, and feed the product into your existing reconstruction function. Nothing in `kspace_core.py` should need to change — if it does, your Milestone-1 interfaces were too narrow.

## C. The PSF analyzer

- Write a function that takes a mask, treats it as if it were a  $k$ -space matrix, and pushes it through your inverse transform (with the proper shift sandwich) to obtain the PSF image.
- Display the PSF's magnitude on a log scale (its side lobes are tiny compared to the main spike — without log scaling you will see nothing).
- Also extract and plot the PSF's central row as a simple 1D curve, so the main spike and the side spikes are unmistakable.
- For the uniform mask, verify the predicted geometry: at acceleration  $R$  you should count exactly  $R$  spikes (main + ghosts) spaced  $N/R$  pixels apart.

## D. The comparison experiment (your Week-2 mini-report)

- Fix  $R = 4$ . Reconstruct the phantom through the uniform, VDS, and random masks. Save a figure with three columns: mask, PSF, reconstruction.
- Write three sentences summarizing what you see: coherent ghosts for uniform, noise-like grain for random, best contrast preservation for VDS.

## How to Test It Before Moving On

- **Identity test:** a mask of all ones must reproduce the original image to within  $10^{-12}$  — undersampling machinery with no undersampling.
- **Ghost-count test:** with uniform  $R = 2$ , the reconstruction must show exactly one extra ghost copy shifted by half the image height. With  $R = 4$ : three ghosts at quarter-image spacings.
- **Direction test:** if your mask skips *rows*, ghosts must appear along the *vertical* axis. If they appear horizontally, your mask is transposed.
- **Energy test:** the masked  $k$ -space must have strictly less total energy than the original (you deleted entries; Parseval says image energy drops accordingly).
- **PSF test:** the PSF of a full mask must be a single spike at the center; its total

energy must not be spread anywhere else.

#### Self-Check: Can You Answer These?

1. Define the acceleration factor  $R$  and explain what a scanner gains and loses by raising it.
2. State the Nyquist criterion as it applies to  $k$ -space line spacing and field of view (Equation 3.1).
3. Why does skipping every second  $k$ -space row cause a *half-image* shift of the ghost, rather than some other shift?
4. Why do random masks produce noise-like artifacts while uniform masks produce crisp ghosts? (Hint: what does regularity in  $k$ -space imply through the convolution theorem?)
5. What is a Point Spread Function, and why does analyzing it let you predict artifacts *without* reconstructing any actual image?
6. Write down the convolution theorem and explain each symbol in Equation (3.5).
7. Why does variable-density sampling preserve image contrast better than uniform sampling at the same  $R$ ? (Hint: Figure 2.2 — where does the energy live?)

# Milestone 3 — Gibbs Ringing, Apodization, and Noise

## 4.1 The problem: sharp scissors leave ripples

### The Concept in Plain English

Every real scan is *finite*: at some radius,  $k$ -space simply stops being measured. Mathematically that is a brutal act — we take the infinite recipe of gratings that a perfectly sharp image needs (remember Figure 2.1: edges need infinitely many high frequencies) and chop it off mid-sentence. The image responds by *ringing*: every sharp edge grows a series of faint parallel ripples beside it, like the rings around a stone dropped into water. This is **Gibbs ringing** (also called truncation artifact), and in clinical images it matters — a ripple next to the spinal cord can mimic a lesion that is not there.

### Analogy

Ending a song by yanking the speaker cable out of the wall produces an ugly “pop.” Ending it with a smooth fade-out does not. The audio content is the same; the difference is entirely in *how abruptly you stop*. Apodization is the fade-out applied to  $k$ -space: instead of frequencies being fully present and then suddenly absent (a cliff), we let them taper gently to zero (a slope).

The 1D picture makes the phenomenon unmistakable. Reconstructing a perfect step edge from only a finite number of frequencies produces overshoot and ripples that *never disappear* no matter how many terms you add — they only get pushed closer to the edge:

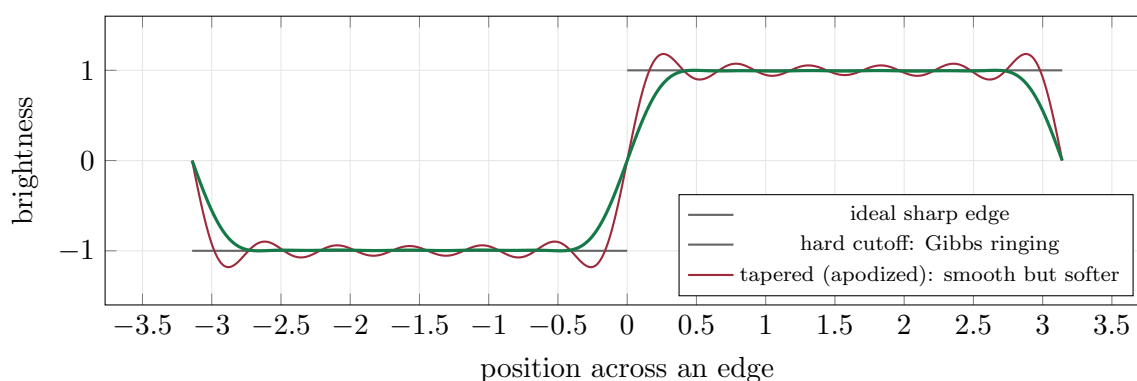


Figure 4.1: Reconstructing a sharp edge from finitely many frequencies. A hard cutoff (red) overshoots and ripples — Gibbs ringing. Tapering the same frequencies smoothly to zero (green) kills the ripples at the cost of a slightly softer edge. This trade-off is the entire story of apodization.

## 4.2 The fix: apodization windows

### The Concept in Plain English

An **apodization filter** (from Greek, literally “removing the foot”) is a smooth weighting matrix  $W[u, v]$ , valued 1.0 at the center of  $k$ -space and gliding down toward 0.0 at the periphery, which we multiply element-wise into  $k$ -space before reconstruction. High frequencies are not amputated — they are *faded*. The result: ringing suppressed, at the price of mild blurring. There is no free lunch; the app’s job is to let the user *feel* this trade-off with a slider.

Two classic window shapes are all you need:

- **2D Hamming window** — built as the product of two 1D raised cosines (one across each axis), for a matrix with centered indices  $u, v$ :

$$W_{\text{Ham}}[u, v] = \left(0.54 + 0.46 \cos \frac{\pi u}{N/2}\right) \left(0.54 + 0.46 \cos \frac{\pi v}{N/2}\right). \quad (4.1)$$

Note it does not descend all the way to zero (it floors at 0.08) — a deliberate design that balances ripple suppression against blur.

- **2D Gaussian window** — the smoothest taper in mathematics, radially symmetric, with a width knob  $\sigma$ :

$$W_{\text{Gauss}}[u, v] = \exp\left(-\frac{u^2 + v^2}{2\sigma^2}\right). \quad (4.2)$$

Small  $\sigma$  = aggressive fade = maximal smoothing and blur; large  $\sigma$  = barely any effect. This  $\sigma$  becomes a GUI slider.

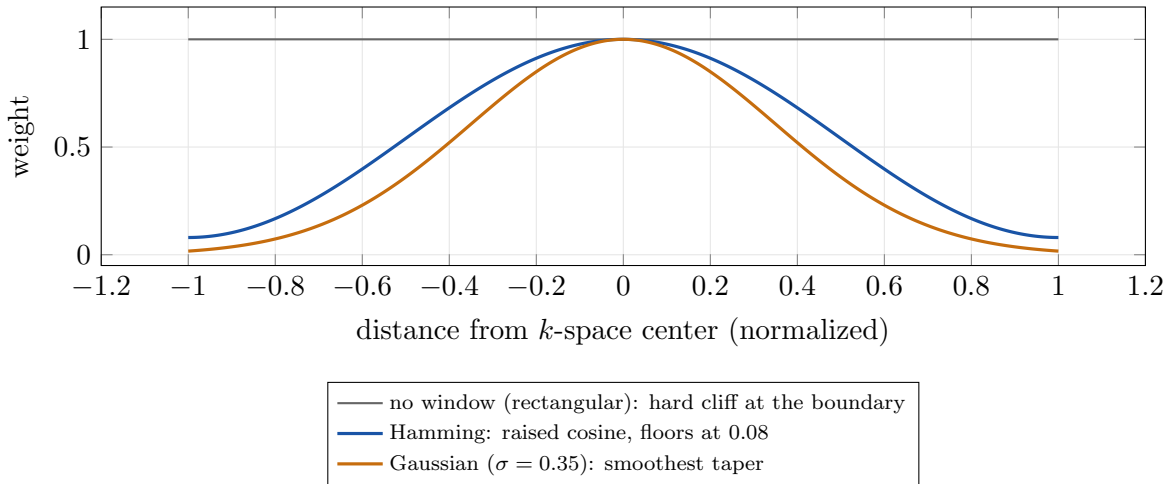


Figure 4.2: Cross-sections of the window options. The rectangular “window” (doing nothing) ends in a cliff — that cliff is what rings. Both smooth windows trade a controlled amount of blur for ringing suppression.

### Why This Matters

Why does multiplying  $k$ -space by a window equal blurring the image? The convolution theorem again (Equation 3.5)! Multiplication in  $k$ -space = convolution in image space. The inverse transform of a Gaussian window is... another Gaussian — a small soft blob. Convoluting the image with a small soft blob is precisely a gentle blur. The same theorem

that explained ghosts in Chapter 3 explains blur here. One theorem, every artifact.

### 4.3 Thermal noise: why real scans are grainy

#### The Concept in Plain English

The MRI signal is a faint radio whisper measured by an antenna, and every warm object — the patient’s body, the receiver electronics — constantly emits random electromagnetic crackle of its own. This **thermal noise** adds itself to every  $k$ -space sample at acquisition time. Crucially, the measurement is complex (magnitude and phase), so the noise strikes *both* the real and imaginary channels, independently:

$$K_{\text{noisy}}[u, v] = K[u, v] + \mathcal{N}(0, \sigma_n^2) + i\mathcal{N}(0, \sigma_n^2), \quad (4.3)$$

where  $\mathcal{N}(0, \sigma_n^2)$  means a random draw from a Gaussian (bell curve) with mean zero and standard deviation  $\sigma_n$ . Because the Fourier transform spreads every  $k$ -space sample across the whole image, the noise does not stay put — it lands *everywhere*, as uniform grain over the entire reconstruction.

Two facts worth knowing (and worth demonstrating in your app):

- **Noise must be added in  $k$ -space, not to the image.** That is where it physically arises. A subtle consequence: after taking the magnitude of the reconstruction, dark regions show a characteristic non-Gaussian “floor” of noise (magnitudes cannot be negative, so noise in black areas piles up above zero — the Rician distribution). Adding noise directly to the image would miss this entirely.
- **Noise and undersampling compound each other.** Fewer samples means less signal averaging, so accelerated scans are noisier in practice — your metrics dashboard (Milestone 5) will show PSNR falling under both sliders at once.

### 4.4 Implementation guide

#### What to Build — Step-by-Step (No Code, Just Instructions)

This milestone lives in `filters.py`. It is deliberately light — a breather between the sampling and gridding milestones.

##### A. Centered coordinate grids (build once, reuse everywhere)

- Write one helper that returns two matrices  $U$  and  $V$  holding, for every entry, its horizontal and vertical offset from the matrix center. You already needed this for the circular mask in Milestone 2 — reuse it, do not rewrite it.

##### B. The window generators

- **Hamming:** build the 1D raised-cosine profile along one axis, build it along the other, and form the 2D window as their outer product. Parameterize by matrix size only.
- **Gaussian:** evaluate the radial Gaussian formula on the  $U, V$  grids. Parameterize by  $\sigma$ , and express  $\sigma$  as a fraction of the matrix half-width so the slider feels natural at any image size.
- Both functions must return matrices of the same shape as  $k$ -space with values in  $[0, 1]$ , maximal at the center.

### C. The noise injector

- Write a function taking a complex  $k$ -space matrix and a noise level  $\sigma_n$ : draw one Gaussian random matrix for the real part, an *independent* one for the imaginary part, combine them into a complex matrix, and add it to the input.
- Accept an optional seed for reproducibility.
- Scale  $\sigma_n$  relative to the mean  $k$ -space magnitude (e.g. the slider sets a percentage), so “noise level 0.05” means something consistent across different input images.

### D. Pipeline placement

- Order matters and should mirror physics: start from full  $k$ -space, *first* add noise (the scanner receives noisy data), *then* apply the sampling mask (the scanner only keeps acquired lines), *then* apply the apodization window (a processing choice made after acquisition), then reconstruct.

### E. The demonstration experiment

- Reconstruct the phantom through a tight circular mask (from Milestone 2) with no window — zoom into a sharp boundary and capture the ripples.
- Repeat with the Hamming window applied. Save the pair side by side: ringing gone, edge slightly softer.
- Reconstruct with three noise levels (e.g.  $\sigma_n$  at 1%, 5%, 15% of mean magnitude) and save the progression from clean to grainy.

## How to Test It Before Moving On

- **Window shape test:** both windows must equal exactly 1.0 at the center entry and decrease monotonically outward along any axis; the Gaussian must be radially symmetric (value depends only on distance from center).
- **Ringing test:** plot a single row of the reconstruction crossing a sharp phantom edge, with and without the window. The un-windowed curve must visibly overshoot and oscillate; the windowed one must not.
- **Noise statistics test:** with the image zeroed out, inject noise and reconstruct; the result’s standard deviation should scale linearly when you double  $\sigma_n$ .
- **Determinism test:** the same seed must produce the identical noisy reconstruction twice.

## Self-Check: Can You Answer These?

1. What causes Gibbs ringing, and near which image features does it appear?
2. Why does adding *more* frequencies never eliminate Gibbs ringing, only narrow it?
3. Explain apodization using the fade-out analogy, then state precisely what is multiplied with what.
4. What is the fundamental trade-off of every apodization window? Which slider in your app exposes it?
5. Use the convolution theorem to explain why a Gaussian window in  $k$ -space blurs the image.
6. Why is MRI noise added to the *complex  $k$ -space* rather than to the final image, and

what visible difference does that choice make in dark image regions?

7. Why does the Hamming window's floor value of 0.08 (rather than 0) represent a design compromise?

# Milestone 4 — Radial & Spiral Trajectories, Density Compensation, and Gridding

This is the most mathematically demanding milestone. Do not start it until Milestones 1–3 pass all their tests. The good news: it decomposes into three small, independent ideas — *drawing paths*, *re-weighting points*, and *snapping points to a grid* — each simple on its own.

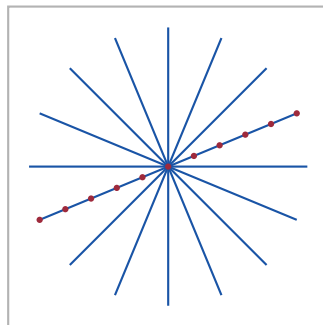
## 5.1 Why scanners abandon straight lines

### The Concept in Plain English

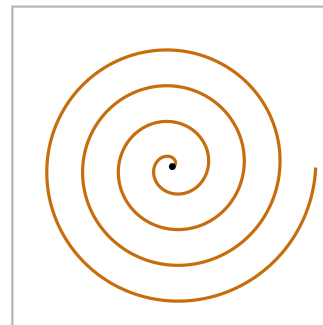
Cartesian scanning reads  $k$ -space like a book: row by row, left to right. But nothing in physics requires straight rows — the scanner’s magnetic gradients can steer the sampling point along *any* smooth path through  $k$ -space. Two alternative families dominate:

- **Radial:** straight spokes through the center, like wheel spokes or the hands of a clock sweeping around. Every spoke passes through the center, so the low frequencies (the contrast) are re-measured on every single spoke. If the patient moves, no single “stripe” of  $k$ -space is ruined — motion errors smear into mild streaks instead of catastrophic ghosts. This is why radial imaging is used on children and cardiac patients.
- **Spiral:** one continuous corkscrew from the center outward. The gradients never slam between extremes, so a spiral covers  $k$ -space extremely fast — popular in functional MRI where speed is everything.

The price for these benefits: the samples no longer land on the neat integer grid that the FFT demands. Fixing that mismatch is the whole engineering content of this milestone.



**Radial:**  $N_{\text{spokes}}$  lines at angles  $\theta_m$   
dots show equal spacing *along* a spoke



**Archimedean spiral:** one continuous path  
 $k_x = at \cos \omega t$ ,  $k_y = at \sin \omega t$

Figure 5.1: The two non-Cartesian trajectories. Neither lands on the integer grid the FFT requires.

## 5.2 Drawing the paths: just trigonometry

**Radial.** A spoke is a straight line through the origin at angle  $\theta$ . Choose  $N_{\text{spokes}}$  angles spread evenly over half a turn (half suffices — a spoke at  $\theta$  and one at  $\theta + 180^\circ$  trace the same line):

$$\theta_m = \frac{m\pi}{N_{\text{spokes}}}, \quad m = 0, 1, \dots, N_{\text{spokes}} - 1. \quad (5.1)$$

Along each spoke, walk a radius variable  $r$  from  $-k_{\text{max}}$  to  $+k_{\text{max}}$  in equal steps and convert polar to Cartesian coordinates — secondary-school trigonometry, nothing more:

$$k_x(r, \theta_m) = r \cos \theta_m, \quad k_y(r, \theta_m) = r \sin \theta_m. \quad (5.2)$$

**Spiral.** An Archimedean spiral grows its radius linearly while the angle turns steadily. With a time-like parameter  $t$ :

$$k_x(t) = a t \cos(\omega t), \quad k_y(t) = a t \sin(\omega t), \quad (5.3)$$

where  $\omega$  sets how fast it turns and  $a$  how fast it grows ( $a$  is chosen so the spiral just reaches  $k_{\text{max}}$  at the final  $t$ ).

*Where do the sample values come from in a simulation?* A real scanner measures them; we do not have a scanner. We take the full Cartesian  $k$ -space from Milestone 1 and **interpolate** (look up in-between values) at each trajectory coordinate — as if the scanner had sampled exactly there. This “forward interpolation” step is unique to simulation and easy to forget.

## 5.3 The density problem: spokes crowd the center

### Analogy

Look at a bicycle wheel. Near the hub, the spokes almost touch each other; at the rim, they are far apart. Radial  $k$ -space sampling has exactly this geometry: the center of  $k$ -space gets sampled by *every* spoke, while each patch of the periphery is visited by only one. If you naively add all samples onto a grid, the center — the low frequencies, the image contrast — gets counted dozens of times over. The reconstruction comes out as a foggy, glowing, contrast-crushed mess: technically an image, but drowning in low-frequency glare.

The cure is **density compensation**: before gridding, multiply each sample by a weight that says “how much area of  $k$ -space do *you* alone represent?” Samples in crowded territory get small weights; lonely peripheral samples get large ones. For radial trajectories the crowding is exactly inversely proportional to radius, so the weight is simply proportional to the radius — the famous **ramp filter**:

$$w(r) = |r| = \sqrt{k_x^2 + k_y^2}. \quad (5.4)$$

### Why This Matters

The most satisfying moment of the milestone: reconstruct radial data *without* density compensation, then flip it on. The image snaps from a blurry glowing fog into a properly contrasted picture, instantly. Put a checkbox for exactly this in your GUI — it is the best five-second demonstration in the entire application.

## 5.4 Gridding: getting scattered points back onto the FFT’s grid

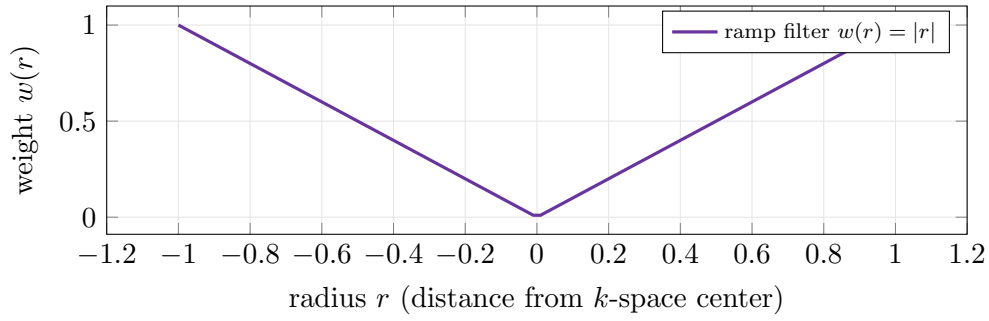


Figure 5.2: The radial density compensation function. Center samples (massively over-represented) are weighted toward zero; edge samples (sparse) are weighted up. Without this, reconstructions are drowned in low-frequency glare.

### The Concept in Plain English

The inverse FFT is a machine with a rigid input socket: it accepts only a full rectangular matrix of values at integer positions. Our weighted trajectory samples are scattered at arbitrary real-valued coordinates like  $(37.42, -101.77)$ . **Gridding** is the adapter between the two: every scattered sample deposits its value onto nearby integer grid cells, and once all samples have been deposited, we have an ordinary matrix and proceed exactly as in Milestone 1.

Three strategies, in increasing order of quality:

1. **Nearest neighbor** — each sample dumps its entire value into the single closest grid cell (round both coordinates). Crude: two samples rounding to the same cell collide, and cells no sample rounds to stay empty. Fine as a first prototype.
2. **Bilinear spreading** — each sample splits its value among the *four* grid cells that surround it, in proportion to closeness. A sample at fractional position  $(\alpha, \beta)$  inside its cell gives the four corners the weights

$$(1-\alpha)(1-\beta), \quad \alpha(1-\beta), \quad (1-\alpha)\beta, \quad \alpha\beta, \quad (5.5)$$

which always sum to 1 — no signal is created or lost. Much smoother results.

3. **Kernel convolution (e.g. Kaiser–Bessel)** — each sample spreads over a small neighborhood ( $4 \times 4$  or more cells) shaped by a carefully designed bump function. This is what professional reconstruction packages do. In this project it is an optional stretch goal; bilinear is fully sufficient for the demonstration.

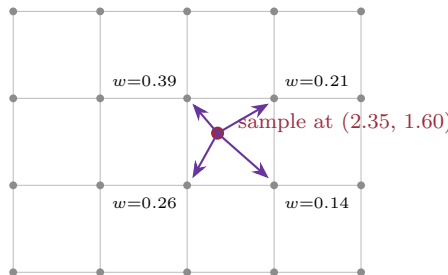


Figure 5.3: Bilinear gridding: one scattered sample (red) splits its value among its four surrounding integer grid cells, weighted by closeness. The four weights sum to exactly 1.

One more bookkeeping detail: keep a parallel **weight-accumulation matrix**. Every time a sample deposits weight  $w$  into a cell, also add  $w$  into the same cell of the weight matrix. At the

end, divide the data grid by the weight grid (wherever the weight is nonzero) — otherwise cells that happened to receive many samples end up artificially bright.

## 5.5 What the artifacts look like now

Undersampled radial data (too few spokes) does *not* produce the clean ghost copies of Cartesian undersampling. Instead the PSF of a spoke pattern is a star shape, so the artifact is **streaks** — fine straight lines radiating across the image, one family per missing angle. Spiral undersampling produces swirling, curved artifacts. Comparing these three artifact families (ghosts vs. streaks vs. swirls) side by side, at matched acceleration, is the signature experiment of your finished application.

## 5.6 Implementation guide

### What to Build — Step-by-Step (No Code, Just Instructions)

Two files: `trajectories.py` (paths and weights) and `gridding.py` (scatter-to-grid). Keep them strictly separate — the trajectory module must know nothing about grids, and the gridding module must work for *any* list of coordinates it is handed.

#### A. Trajectory generation (`trajectories.py`)

- Radial: given  $N_{\text{spokes}}$  and the number of samples per spoke, produce the list of  $(k_x, k_y)$  coordinates — compute the evenly spaced angles over  $[0, \pi)$ , walk  $r$  symmetrically from  $-k_{\text{max}}$  to  $+k_{\text{max}}$ , and apply the cosine/sine conversion. Return flat arrays of coordinates.
- Spiral: given a total number of samples and number of turns, generate the parameter  $t$ , apply the Archimedean formulas, and choose the growth constant so the final radius equals  $k_{\text{max}}$ .
- Density weights: for radial, return  $w = \sqrt{k_x^2 + k_y^2}$  for every sample. (For the spiral, the same radial ramp is an acceptable approximation at this project’s level — note it as such in your report.)
- Add a quick visual: a scatter plot of the generated coordinates, colored by weight, so mistakes in the math are visible instantly.

#### B. Simulated acquisition (the forward direction)

- Write the “pretend scanner”: given the full Cartesian  $k$ -space from Milestone 1 and a list of trajectory coordinates, interpolate the complex  $k$ -space values at those exact coordinates (use a library interpolator; note that complex data may require interpolating real and imaginary parts separately).

#### C. The gridding engine (`gridding.py`)

- **Prototype first** with the library scattered-interpolation routine (`griddata`-style): hand it the coordinates, the density-weighted values, and the target integer grid; ask for linear interpolation with zero fill outside the sampled region. This gets you a working end-to-end radial reconstruction in one afternoon.
- **Then** implement your own bilinear spreader (this is the part worth marks): allocate a zero complex matrix and a zero weight matrix; for each sample, find the four surrounding integer cells, compute the four closeness weights, deposit weighted value and weight; finally divide data by weights where weights are nonzero.

- Guard the edges: samples whose neighborhood falls outside the matrix must be clipped or skipped, never allowed to wrap around.
- After gridding, the matrix is ordinary centered  $k$ -space — reconstruct with the exact same inverse-transform function from Milestone 1.

#### D. The signature experiments

- The density-compensation reveal: same radial data reconstructed with weights off vs. on. Save both.
- The spoke-count sweep: reconstruct at (say) 16, 32, 64, 128, 256 spokes and watch streaks fade as spokes increase.
- The artifact-family comparison: Cartesian  $R=4$  ghosts vs. radial streaks vs. spiral swirls, at matched sample counts, side by side.

#### How to Test It Before Moving On

- **Coordinate sanity:** every radial spoke's samples must lie on a straight line through the origin; the maximum radius must equal  $k_{\max}$ ; the scatter plot must look like Figure 5.1.
- **Weight conservation:** in your bilinear spreader, the four weights for any sample must sum to 1 within floating-point error.
- **Dense-limit test:** with very many spokes (e.g.  $\geq 2N$ ) and density compensation on, the radial reconstruction must closely match the fully sampled Cartesian reconstruction (small residual error is expected from interpolation, but the images should be visually indistinguishable).
- **Glare test:** switching density compensation *off* must produce the characteristic foggy low-frequency glare. If nothing changes, your weights are not actually being applied.
- **Streak test:** at 32 spokes you must see radiating straight streaks — not vertical ghost copies. Ghost copies mean you accidentally ran a Cartesian mask.

#### Self-Check: Can You Answer These?

1. Give two clinical reasons a scanner might prefer radial acquisition over Cartesian.
2. Why can radial angles be confined to  $[0, \pi)$  instead of a full turn?
3. Explain, with the bicycle-wheel picture, why uncompensated radial data reconstructs with crushing low-frequency glare.
4. Why is the radial density weight exactly proportional to  $|r|$ ?
5. Why can the inverse FFT not be applied directly to trajectory samples? What property of its input does it demand?
6. Walk through what happens to a single sample at coordinates (10.3, 20.8) in nearest-neighbor gridding vs. bilinear gridding.
7. Why must you accumulate a separate weight matrix and divide by it at the end?
8. Why do too-few spokes produce *streaks* while skipped Cartesian rows produce *ghost copies*? (Answer via the shape of each pattern's PSF.)

# Milestone 5 — Measuring Image Quality: MSE, PSNR, and SSIM

## 6.1 From “it looks worse” to numbers

### The Concept in Plain English

Every previous milestone *damages* images in some controlled way. This milestone asks: *by how much, exactly?* A simulator that only shows pictures teaches intuition; a simulator that also shows **numbers** teaches science — because numbers can be plotted against the acceleration slider, compared across sampling strategies, and quoted in your report. We use three standard metrics, each answering a subtly different question, plus one visual tool (the error heatmap) that shows *where* the damage lives.

Throughout,  $f$  is the ground-truth original image and  $\hat{f}$  the reconstruction, both  $N \times M$  and normalized to  $[0, 1]$ .

## 6.2 MSE: the average squared mistake

$$\text{MSE} = \frac{1}{NM} \sum_{m=0}^{N-1} \sum_{n=0}^{M-1} |f[m, n] - \hat{f}[m, n]|^2. \quad (6.1)$$

Read it literally: at every pixel, take the error; square it (so positive and negative errors cannot cancel, and big errors are punished disproportionately); average over all pixels. MSE is simple, fast, and the building block of PSNR — but it is *blind to structure*: shifting an image by one pixel barely changes what a human sees yet can produce a large MSE, while a smooth global brightness drift can produce the same MSE as ugly localized ghosting.

## 6.3 PSNR: MSE on a decibel scale

$$\text{PSNR} = 10 \log_{10} \left( \frac{\max(f)^2}{\text{MSE}} \right) \quad [\text{dB}]. \quad (6.2)$$

PSNR repackages MSE as a ratio: *how large is the strongest possible signal compared to the average error?* — expressed in decibels because the logarithm matches how quality differences feel (each  $\sim 6$  dB step roughly halves the error amplitude). Interpretive anchors for your dashboard: above  $\sim 40$  dB reconstructions look essentially perfect; 30–40 dB shows mild visible degradation; below  $\sim 25$  dB the damage is obvious. Note the direction: **higher PSNR is better**, and PSNR is infinite for a perfect reconstruction ( $\text{MSE} = 0$ ) — your code must handle that division specially.

## 6.4 SSIM: asking what a human would say

MSE and PSNR compare pixels in isolation. Human vision does not — it compares *neighborhoods*: local brightness, local contrast, and local structure. The **Structural Similarity Index** formalizes exactly those three comparisons. For each small sliding window of the two images it

computes local means  $\mu_f, \mu_{\hat{f}}$ , local variances  $\sigma_f^2, \sigma_{\hat{f}}^2$ , and the local covariance  $\sigma_{f\hat{f}}$  (do the two windows brighten and darken *together?*), then combines them:

$$\text{SSIM}(f, \hat{f}) = \frac{(2\mu_f\mu_{\hat{f}} + C_1)(2\sigma_{f\hat{f}} + C_2)}{(\mu_f^2 + \mu_{\hat{f}}^2 + C_1)(\sigma_f^2 + \sigma_{\hat{f}}^2 + C_2)}. \quad (6.3)$$

- The first factor compares **luminance**: it equals 1 exactly when the local mean brightnesses agree.
- The second factor compares **contrast and structure** through the covariance: it is large only when the two windows vary *in the same pattern*.
- $C_1$  and  $C_2$  are tiny constants that prevent division by zero in flat regions — stabilizers, not physics.
- The final score averages over all windows, landing in  $[-1, 1]$ : 1 means structurally identical, values near 0 mean structurally unrelated.

### Why This Matters

Why carry three metrics instead of one? Because they *disagree*, and the disagreements are informative. A mild global blur can keep SSIM decently high (structure mostly intact) while PSNR drops sharply; strong noise does the reverse (structure destroyed, average error moderate). When your dashboard shows all three side by side, the user learns not just *that* quality dropped, but *what kind* of quality dropped — which is precisely the pedagogical point of the simulator.

## 6.5 The error heatmap: seeing *where* it went wrong

The fourth panel of your GUI displays the absolute difference image

$$E[m, n] = |f[m, n] - \hat{f}[m, n]| \quad (6.4)$$

rendered with a colormap (dark = accurate, bright/hot = wrong). Every artifact family paints its own signature: ghosts appear as displaced copies of the anatomy’s outline; Gibbs ringing as bright fringes hugging edges; radial streaks as a fan of lines; noise as uniform speckle. The heatmap is the app’s best debugging tool, too — one glance usually tells you *which* stage of the pipeline misbehaved.

## 6.6 Implementation guide

### What to Build — Step-by-Step (No Code, Just Instructions)

This lives in `metrics.py`, and it is the easiest module of the project — a good week to catch up on anything that slipped.

- Write the MSE function directly from its formula: element-wise difference, square of magnitudes, mean. Two lines of logic.
- Write PSNR on top of MSE. Handle the perfect-reconstruction case explicitly: if MSE is zero, return infinity (or display “ $\infty$  dB”) rather than dividing by zero.
- Do *not* hand-implement SSIM — the sliding-window bookkeeping is error-prone and `scikit-image` ships a reference implementation. Call it, passing the correct `data_range` argument (the difference between your images’ maximum and minimum

possible values — required because your images are floats in  $[0, 1]$ , not 8-bit integers).

- Write the error-map function: absolute difference matrix, returned as-is; let the display layer choose the colormap (a perceptually ordered one like viridis or inferno — avoid the rainbow “jet” colormap, which fabricates visual edges where none exist).
- Bundle everything into one convenience call that takes  $(f, \hat{f})$  and returns all three numbers plus the error map in a single dictionary — this becomes the one call the GUI makes after every reconstruction.
- **Consistency guard:** metrics are only meaningful if both images share the same scale. Assert both inputs are in  $[0, 1]$  (or normalize both identically) before computing anything — silent scale mismatches are the classic source of nonsense PSNR values.

### How to Test It Before Moving On

- **Identity test:** comparing any image with itself must yield  $\text{MSE} = 0$ ,  $\text{PSNR} = \infty$ ,  $\text{SSIM} = 1.0$  exactly.
- **Known-value test:** compare an all-black image (0.0) with an all-mid-gray image (0.5): MSE must equal exactly 0.25.
- **Monotonicity test:** reconstruct the phantom at  $R = 1, 2, 4, 8$  with the uniform mask; PSNR and SSIM must decrease monotonically as  $R$  grows. If a higher  $R$  ever scores *better*, something is broken.
- **Sensitivity ordering:** add slight blur to one copy and strong noise to another with matched MSE; verify SSIM ranks the blurred copy higher — confirming SSIM captures structure, not just error size.

### Self-Check: Can You Answer These?

1. Why does MSE square the pixel differences instead of just averaging them?
2. What question does PSNR answer that raw MSE does not, and why is a logarithmic (decibel) scale appropriate?
3. Give a concrete example where two distortions have equal MSE but very different visual quality. Which metric detects the difference?
4. What three local quantities does SSIM compare, and what role do the constants  $C_1, C_2$  play?
5. Why is passing the correct `data_range` essential when computing SSIM on floating-point images?
6. You look at an error heatmap and see bright fringes tracing every sharp edge. Which artifact is this, and which milestone’s tool fixes it?
7. Which direction is “better” for each metric — and what are the best-possible values of all three?

# Milestone 6 — The Interactive GUI and Application Architecture

## 7.1 From engine to instrument

### The Concept in Plain English

Everything so far is an *engine*: functions you call from scripts. This milestone wraps it in an *instrument*: a window where dragging a slider re-runs the pipeline and four panels repaint instantly. The teaching power of the whole project lives here — the moment a student drags the acceleration slider and *watches* ghosts crawl into the image while the PSNR readout falls, the abstract mathematics becomes physical intuition.

Desktop GUIs are **event-driven**: the program does not run top to bottom; it sits in an idle loop waiting for events (a slider moved, a dropdown changed), and each event triggers a function you registered for it (a *callback* or *slot*). Your entire job is: build the layout once, connect each control to one callback, and make that callback run the pipeline and refresh the panels.

## 7.2 The layout

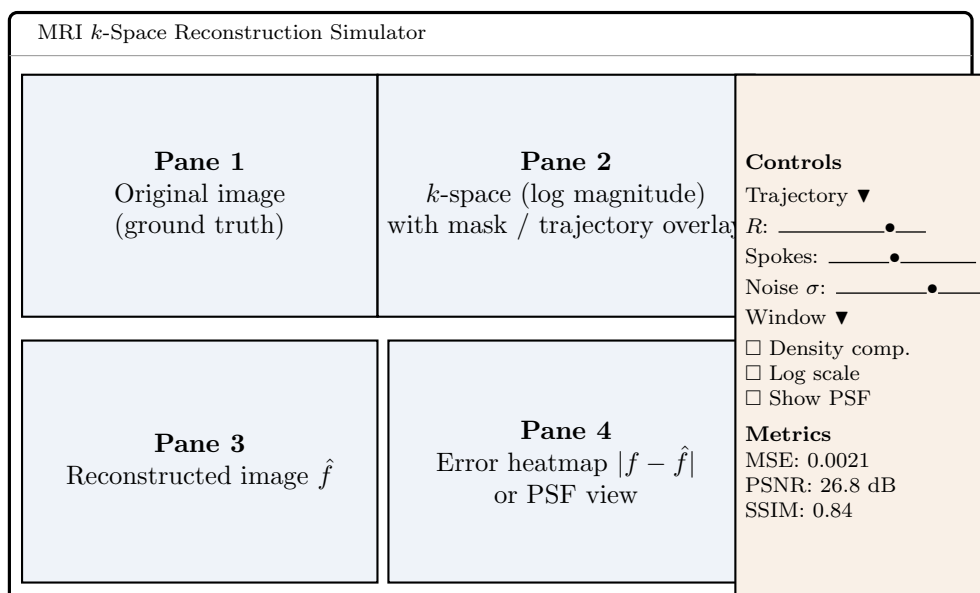


Figure 7.1: The dashboard wireframe: four synchronized viewing panes plus a control column with live metric readouts.

Each pane is a plotting canvas embedded in the window (Matplotlib provides a Qt widget for exactly this). The mask overlay in Pane 2 is what visually links cause to effect: the user *sees* which *k*-space lines survive, right next to the damage they cause.

## 7.3 The one problem that ruins naive GUIs: freezing

### Analogy

A GUI has a single “receptionist” — the **main thread** — whose job is answering events: repainting panels, responding to drags, keeping the window alive. If the receptionist personally runs your reconstruction (gridding alone can take a noticeable fraction of a second), the front desk goes unstaffed: the window stops repainting, the slider freezes mid-drag, and the operating system grays the app out as “Not Responding.” The fix is obvious once stated: the receptionist *delegates*. Heavy computation is handed to a background **worker thread**; the receptionist keeps answering events; when the worker finishes, it sends a message (in Qt, a **signal**) back, and only the final lightweight repaint happens at the front desk.

Two rules keep threaded GUIs sane, and both are iron laws:

1. **Only the main thread touches widgets.** The worker computes pure data (matrices, numbers) and communicates results exclusively through the signal mechanism, which safely crosses the thread boundary.
2. **Stale results must be discarded.** A user dragging a slider fires dozens of events per second. Do not queue a pipeline run for each — either drop new requests while a worker is busy and re-run once at the final value, or cancel the in-flight worker. Otherwise the display lags behind the slider by many seconds, playing back obsolete frames.

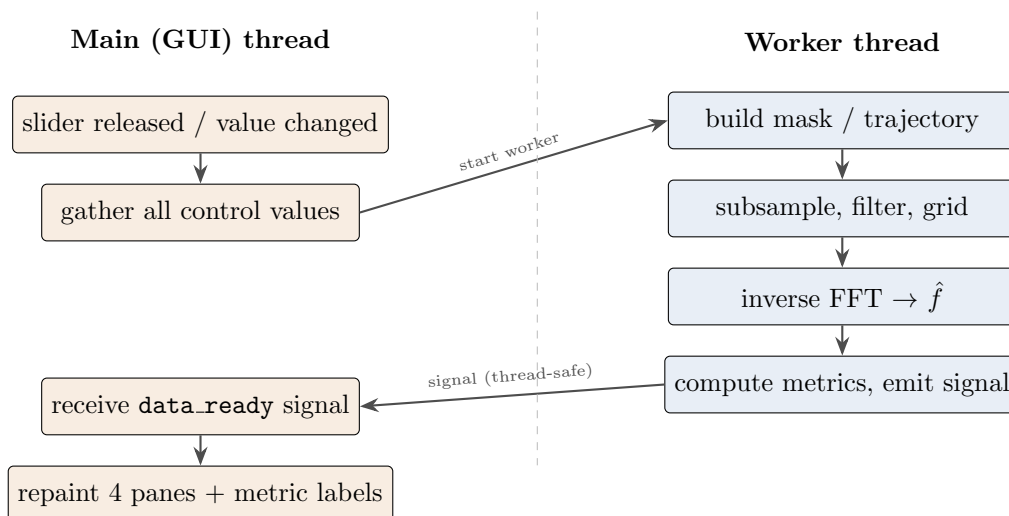


Figure 7.2: The asynchronous control flow. The main thread never computes; the worker thread never touches widgets. They communicate only via signals.

## 7.4 The architecture that keeps six weeks of code sane

The application follows a three-layer separation:

- **View** (the GUI file): windows, panes, sliders. Knows nothing about Fourier transforms — it only collects settings and displays matrices it is given.
- **Pipeline engine** (one coordinating class): receives a settings bundle (trajectory type,  $R$ ,  $\sigma_n$ , window choice, checkbox states), calls your Milestone 1–5 modules in the correct order,

and returns a results bundle (four display matrices + three metrics). This is the *only* code that knows the pipeline’s order of operations.

- **Engine modules** (everything you already built): pure mathematical functions with no idea a GUI exists.

This is why the roadmap insisted on terminal-first development: because your engine modules never import anything GUI-related, the GUI is a thin, optional skin — and every bug can be reproduced in a five-line script without clicking anything.

## 7.5 Implementation guide

### What to Build — Step-by-Step (No Code, Just Instructions)

This lives in `app.py` (plus, ideally, a small `pipeline.py` for the coordinating engine class). Choose **PyQt6** if you want professional threading support (recommended), or `CustomTkinter` for a gentler learning curve.

#### A. The pipeline engine (build this before any window)

- Define a simple settings container listing every user-adjustable parameter: trajectory type, acceleration factor, spoke count, noise level, window type and width, and the checkbox flags.
- Write one method: settings in  $\rightarrow$  results out. Internally it selects mask or trajectory, applies noise, subsamples or grids, filters, reconstructs, and computes metrics — entirely by calling your existing modules.
- Test it from a script: fabricate a settings object, print the metrics. No window needed yet. Cache anything reusable across calls (the input image’s full  $k$ -space does not change when only  $R$  changes — recompute masks, not FFTs).

#### B. The static layout

- Build the main window with a grid layout: four embedded plot canvases in a  $2 \times 2$  arrangement and a vertical control column on the right, as in Figure 7.1.
- Add the controls: a trajectory dropdown; sliders for  $R$ , spoke count, noise level, and window width; checkboxes for density compensation, log scaling, and error-map/PSF toggle; and three read-only metric labels.
- First run the window with all panes showing static placeholder images — confirm the layout resizes sensibly before wiring any logic.

#### C. Wiring events (single-threaded first)

- Connect every control’s change event to one shared callback that reads all current settings, calls the pipeline engine, and pushes the four result matrices into the panes and the three numbers into the labels.
- Accept that the UI stutters during drags at this stage — correctness first.
- Overlay the sampling mask on the  $k$ -space pane (e.g. dim the skipped lines) so cause and effect sit side by side.

#### D. Going asynchronous

- Move the pipeline call into a worker object running on a background thread. The callback now only *launches* the worker with the current settings.
- Define a completion signal carrying the results bundle; connect it to a main-thread method that does nothing but repaint panes and labels.

- Add the busy-guard: if a worker is already running when a new event arrives, remember only the *latest* requested settings and run them when the worker frees up; drop everything in between.
- Verify the iron laws: no widget is touched from the worker, and dragging a slider rapidly never queues a backlog.

#### E. Comfort features (only after everything above works)

- A file-open dialog for loading the user’s own images (through your Milestone 1 loader, so grayscale conversion and resizing are automatic).
- A phantom button that restores the Shepp–Logan test image.
- A save-figure button exporting the current four-pane view to PNG — this is how you will generate every figure in your final report.

### How to Test It Before Moving On

- **Responsiveness test:** drag the  $R$  slider rapidly back and forth. The window must keep repainting, never gray out, and settle on the correct final image within a moment of release.
- **Consistency test:** set specific settings in the GUI, then run the identical settings through the pipeline engine in a plain script; the metrics must match exactly. (This proves the GUI is a pure skin.)
- **Coherence test:** at uniform  $R=2$ , Pane 2 must visibly show half its lines dimmed *and* Pane 3 must show ghosting *and* the PSNR label must drop — all three simultaneously.
- **Stale-frame test:** drag a slider quickly, stop, and count updates: the display must jump to the final value, not replay every intermediate position for seconds afterward.
- **Cross-feature test:** radial trajectory + density compensation checkbox off/on must reproduce the glare-vs-contrast flip from Chapter 5, live.

### Self-Check: Can You Answer These?

1. What does “event-driven” mean, and how does it differ from a script that runs top to bottom?
2. Why does running the reconstruction inside the slider callback freeze the window? Explain using the receptionist analogy, then using the terms “main thread” and “event loop.”
3. State the two iron laws of GUI threading and what goes wrong when each is violated.
4. Why is it valuable that your engine modules never import GUI code? Name two concrete benefits.
5. What is a signal/slot connection, and why is it the *only* safe channel from worker to widgets?
6. A user drags the noise slider through 40 positions in one second. What should your application compute, and what should it skip?
7. Which computations are worth caching between slider events, and which must be redone every time?

# The Build Plan — Order of Work, Edge Cases, and Polish

## 8.1 Why this exact order

The six-week sequence is not arbitrary — each stage produces the test harness for the next:

| Week | Build                           | Why now                                                                                                                                                                      |
|------|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | Fourier engine (Ch. 2)          | Everything else multiplies into, or inverts out of, this. Its round-trip test protects all later work.                                                                       |
| 2    | Masks + PSF (Ch. 3)             | Needs only element-wise multiplication with Week 1's output. First dramatic artifacts appear — strong motivation early.                                                      |
| 3    | GUI skeleton (Ch. 7)            | Contrary to instinct, the GUI comes <i>before</i> the hardest math: watching sliders change images makes Weeks 4–5 debuggable by eye, and spreads GUI learning across weeks. |
| 4    | Filters + noise (Ch. 4)         | Small, self-contained additions to an already-working interactive loop.                                                                                                      |
| 5    | Trajectories + gridding (Ch. 5) | The hardest milestone, attempted when the surrounding scaffolding (visual feedback, metrics, phantom) is mature and trustworthy.                                             |
| 6    | Polish, tests, docs             | Optimization and packaging are only meaningful once behavior is complete and frozen.                                                                                         |

The single most important discipline: **never start milestone  $n+1$  while milestone  $n$ 's tests fail.** Every chapter's test box is a gate, not a suggestion. A bug that slips past its own milestone costs triple to find later, because by then three other modules are suspects.

## 8.2 Edge cases you must handle (Week 6 checklist)

### What to Build — Step-by-Step (No Code, Just Instructions)

#### Input robustness

- **Non-square images:** zero-pad the shorter dimension to a square *before* resizing to  $256 \times 256$ , so anatomy is not stretched. Padding with zeros is harmless in  $k$ -space terms (it merely interpolates the spectrum).
- **Color inputs:** route every loaded image through the luminance conversion; never assume single-channel.
- **Extreme value ranges:** an all-black input, or a constant image, must not crash the normalizer (guard the divide-by-max) or the metrics (guard  $\text{MSE} = 0$  in PSNR).
- **Slider extremes:**  $R=1$  must reproduce the perfect round-trip; the maximum  $R$ , the minimum spoke count, and  $\sigma_n=0$  must all behave sensibly rather than erroring.

#### Performance (only where measured, never speculative)

- Profile first: run the pipeline under a profiler and find where time actually goes. In practice the gridding loop dominates; the FFTs almost never do.
- Vectorize the gridding spreader: replace the per-sample Python loop with array operations that compute all four neighbor indices and weights for *all* samples at once, then deposit with index-based accumulation.
- Cache what does not change: the input image's full  $k$ -space, the coordinate grids, and each mask for given parameters.

#### Packaging and the report

- Repository layout: one folder for engine modules, one for tests, one for the GUI, a README with installation and usage, and a folder of generated result figures.
- Turn every test box in this guide into an automated test file — they are already specified as pass/fail conditions.
- The report writes itself from your experiments: the artifact-family comparison (Ch. 5), the PSNR-vs- $R$  curves per trajectory (Ch. 6), the density-compensation reveal, and the ringing-vs-window study (Ch. 4).
- Record the demo video last, following one narrative arc: full sampling → accelerate → ghosts appear → switch strategies → artifacts change character → metrics quantify everything.

## 8.3 When you get stuck: the debugging ladder

Work down this list; it resolves the overwhelming majority of issues in this project:

1. **Checkerboard pattern over the image?** A missing `fftshift/iftftshift` — audit the sandwich rule of Chapter 2.
2. **Completely black or white display?** A scaling problem — check whether you are displaying raw magnitudes without log compression, or comparing images on different value ranges.
3. **Artifact along the wrong axis?** A transposed mask or swapped row/column indexing.
4. **Radial reconstruction is glowing fog?** Density compensation is missing or not actually multiplied in.
5. **Reconstruction looks right but metrics are absurd?** Scale mismatch — one image in  $[0, 1]$ , the other in  $[0, 255]$ .
6. **GUI freezes?** Computation on the main thread — revisit Figure 7.2.

# Reference: Glossary and Examiner's Question Bank

## 9.1 Glossary — every intimidating term, in one sentence each

### ***k*-space**

The image rewritten as a table of spatial frequencies: each entry records the strength and shift of one striped wave pattern (grating) present in the image.

### **Spatial frequency**

How rapidly brightness oscillates across space — wide lazy stripes are low frequency, tight thin stripes are high frequency.

### **DC component**

The zero-frequency entry at the center of shifted *k*-space; it equals the sum of all pixels and represents average brightness.

### **FFT / IFFT**

Fast algorithms computing the discrete Fourier transform (image → recipe) and its inverse (recipe → image), exactly and losslessly.

### **fftshift / ifftshift**

Quadrant swaps moving the DC term to the matrix center for human-friendly ordering, and back again.

### **Magnitude / Phase**

The two facts stored by each complex *k*-space entry: how strong its grating is, and how far sideways it is shifted.

### **Nyquist criterion**

To represent a signal faithfully you must sample at least twice per cycle of its highest frequency; violating it causes aliasing.

### **Field of view (FOV)**

The physical extent an image can represent; inversely proportional to the spacing of *k*-space samples.

### **Acceleration factor *R***

Fraction of *k*-space lines actually acquired, expressed as a speed-up: *R*=4 means one line kept in four.

### **Aliasing / ghosting / wrap-around**

The fold-over artifact when the effective FOV becomes smaller than the object: shifted copies of the anatomy overlap the image.

### **Sampling mask**

A matrix of ones and zeros multiplied into *k*-space to simulate which samples the scanner acquired.

### **Variable-density sampling**

A mask that keeps the energy-rich center of *k*-space fully and thins only the periphery.

**Point spread function (PSF)**

The image a single bright dot becomes after passing through a sampling pattern; every artifact is the PSF stamped onto every pixel.

**Convolution theorem**

Multiplication in  $k$ -space is convolution in image space (and vice versa) — the single theorem explaining ghosts, blur, ringing, and streaks alike.

**Gibbs ringing**

Ripples beside sharp edges caused by abruptly truncating high frequencies.

**Apodization**

Multiplying  $k$ -space by a smooth window (Hamming, Gaussian) that fades high frequencies gradually, trading ringing for mild blur.

**AWGN**

Additive white Gaussian noise — the random thermal crackle added independently to the real and imaginary channels of every  $k$ -space sample.

**Radial / spiral trajectory**

Non-Cartesian  $k$ -space paths (wheel spokes; a corkscrew) offering motion robustness and speed at the cost of grid mismatch.

**Density compensation function (DCF)**

Per-sample weights (for radial:  $w = |r|$ , the ramp filter) correcting the over-representation of the crowded  $k$ -space center before gridding.

**Gridding**

Depositing scattered trajectory samples onto the integer grid the FFT demands — via nearest-neighbor, bilinear, or kernel spreading.

**Streak artifact**

The radial-undersampling counterpart of ghosting: straight lines fanning across the image, born from the star-shaped PSF of a spoke pattern.

**MSE / PSNR / SSIM**

Quality metrics: average squared pixel error; the same on a decibel scale relative to peak signal; and a perceptual score comparing local luminance, contrast, and structure.

**Parseval's theorem**

Total energy is identical (up to a fixed constant) in image space and  $k$ -space — the Fourier transform loses nothing.

**Shepp–Logan phantom**

The standard synthetic head-like test image used throughout tomography and MRI research.

**Event loop / callback**

The idle cycle in which a GUI waits for user events, and the function invoked when one occurs.

**Worker thread / signal**

A background lane for heavy computation, and the thread-safe message it sends back to the GUI when done.

## 9.2 The examiner's question bank

These are the questions an instructor is most likely to ask at a project demonstration, with compact model answers. If you can deliver each answer in your own words, you are ready.

## Foundations

### Q: Why does MRI need the Fourier transform at all?

The scanner physically measures spatial frequencies (radio signals encoded by magnetic gradients), not pixel brightnesses; the inverse Fourier transform is the mathematical bridge from the measured frequency table to the anatomical image.

### Q: What would the $k$ -space of a perfectly uniform gray image look like?

A single nonzero value at the DC center and zero elsewhere — a constant image contains only the zero-frequency “flat sheet” grating.

### Q: Why is $k$ -space complex-valued?

Each grating needs two numbers recorded — amplitude and sideways shift — and a complex number’s magnitude and phase store exactly those two.

### Q: Where is most of the energy in a natural image’s $k$ -space, and why?

Concentrated in the low-frequency center, because natural images are mostly smooth regions with edges being spatially rare.

## Sampling and artifacts

### Q: Derive what happens when you keep only every second $k$ -space row.

Doubling the line spacing  $\Delta k$  halves the FOV ( $\text{FOV} = 1/\Delta k$ ); the object no longer fits and wraps, producing a ghost shifted by exactly half the image. Equivalently: the mask’s PSF has two spikes half an image apart, and convolution stamps both.

### Q: Why are random-mask artifacts noise-like rather than ghost-like?

Ghosts require the PSF’s side energy concentrated into discrete spikes, which only *periodic* masks produce; a random mask spreads its PSF side energy incoherently, so the artifact looks like grain.

### Q: You increase $R$ and PSNR barely drops for VDS but plummets for uniform sampling — why?

VDS always preserves the fully sampled low-frequency center holding nearly all the image energy and contrast, so the surviving errors touch only low-energy detail frequencies.

### Q: What is the PSF of a *full* sampling mask?

A single perfect spike — convolution with it changes nothing, hence artifact-free reconstruction.

## Filtering and noise

### Q: Why can’t Gibbs ringing be fixed by acquiring more frequencies?

Truncation at *any* finite radius causes overshoot of a fixed relative height (about 9% for an ideal edge); more frequencies only squeeze the ripples closer to the edge. Only a *smooth* taper removes them.

### Q: Prove (informally) that windowing blurs.

By the convolution theorem, multiplying  $k$ -space by a window equals convolving the image with the window’s inverse transform — a small smooth blob — and convolution with a blob is blurring by definition.

### Q: Why add noise to $k$ -space instead of the image?

Because that is where the physics injects it: the antenna measures each complex sample through noisy electronics. It also reproduces the correct Rician noise floor in dark regions after magnitude reconstruction, which image-domain noise would not.

## Non-Cartesian imaging

**Q: State the two problems radial data poses for FFT reconstruction and the fix for each.**

(1) Samples are non-uniformly dense — the center is oversampled — fixed by density-compensation weights  $w = |r|$ . (2) Samples miss the integer grid — fixed by gridding (interpolating/spreading onto the grid).

**Q: Why exactly is the radial weight proportional to  $|r|$ ?**

Spokes are angularly equidistant, so the area each sample represents is an annular wedge whose area grows linearly with radius; weighting by  $|r|$  restores equal representation per unit of  $k$ -space area.

**Q: Why do the four bilinear weights have to sum to one?**

Each sample must deposit exactly its own measured signal — no more, no less; weights summing to one conserve the total signal during spreading.

**Q: Why streaks instead of ghosts?**

The artifact is always the PSF's shape. A spoke pattern's inverse transform is star-shaped, so its side energy lies along lines — streaks; a periodic row pattern's PSF is a spike train — ghosts.

## Metrics and software

**Q: Two reconstructions have identical MSE; SSIM says one is far better. What might they look like?**

One mildly and uniformly blurred (structure preserved — high SSIM), the other with the same error budget spent on localized ghosts or heavy noise (structure disrupted — low SSIM).

**Q: Why does your GUI stay responsive during heavy recomputation?**

Reconstruction runs on a worker thread while the main thread keeps servicing the event loop; results return via a thread-safe signal, and only the lightweight repaint runs on the main thread.

**Q: How do you know your whole pipeline is correct?**

Layered evidence: exact round-trip and Parseval tests for the transform core; predicted-vs-observed ghost geometry for masks; weight conservation and the dense-limit test for gridding; identity and monotonicity tests for metrics; and GUI-vs-script consistency for the application shell.

### Key Takeaways

You now hold the full map: five ideas (matrix, recipe, damage, re-gridding, measurement), one theorem doing most of the work (convolution), one golden rule (the shift sandwich), and one discipline (test every milestone before the next). Build it in order, test at every gate, and the “overwhelming” roadmap becomes six calm weeks of matrix arithmetic. Good luck — and enjoy the moment the first ghost crawls across your screen. You will know exactly why it is there.

# Team Onboarding — Environment Setup and the Project Map

## The Concept in Plain English

This chapter exists so that any teammate can go from a fresh laptop to “running the app and editing code” in under fifteen minutes, without asking anyone for help. It has two halves: first the **environment setup** (the exact commands, in order, with the failures you are most likely to hit), then the **project map** — a guided tour of every file and folder, what lives in it, which milestone it belongs to, and the one architectural rule that keeps the codebase from collapsing.

## 10.1 What must already be on your machine

- **Python 3.12 (64-bit)**. Not 3.13, and especially not the “free-threaded” 3.13t build — PySide6 and `scipy` do not ship reliable wheels for it yet. Check with `py --list`; if 3.12 is missing, install it from [python.org](https://python.org) (tick *Add python.exe to PATH* during install).
- **Git**, for cloning and pushing.
- **VS Code** with the Microsoft *Python* extension (or any editor you prefer — but the steps below assume VS Code).

## 10.2 Setup, step by step

All commands are for **PowerShell**, run from the project root (the folder containing `app.py`).

### Step 1 — Get the code

```
git clone <repository-url>
cd log_synthesis
```

### Step 2 — Create the virtual environment, pinned to 3.12

```
py -3.12 -m venv .venv
```

## Why This Matters

A *virtual environment* is a private copy of Python that lives inside the project folder. Packages installed into it cannot break any other project on your machine, everyone on the team gets the same versions, and deleting the `.venv` folder undoes everything. It is the standard way Python teams avoid “works on my machine.”

### Step 3 — Activate it

```
.\.venv\Scripts\Activate.ps1
```

Your prompt should now begin with `(.venv)`. Two common failures:

- “*running scripts is disabled on this system*” — run `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass` first (affects only the current window), then activate again.
- `python --version` prints something other than 3.12.x — activation did not take; open a fresh terminal and redo this step.

**You must activate in every new terminal.** Forgetting this is the number-one cause of “but I installed numpy, why `ModuleNotFoundError`?”

## Step 4 — Install the dependencies

```
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

The five packages and their jobs:

| Package      | What it is for                                           |
|--------------|----------------------------------------------------------|
| numpy        | all matrix math and the FFT (Milestone 1)                |
| scipy        | scattered-point interpolation for gridding (Milestone 5) |
| scikit-image | Shepp–Logan phantom and the SSIM metric                  |
| matplotlib   | drawing images into the four GUI panels                  |
| PySide6      | the desktop window, sliders, and threading               |

If you ever add a package, immediately regenerate the manifest so the rest of the team gets it too:

```
python -m pip freeze > requirements.txt
```

## Step 5 — Point VS Code at the venv

Open the command palette (`Ctrl+Shift+P`), run *Python: Select Interpreter*, and choose the entry showing `.venv\Scripts\python.exe`. This is separate from terminal activation — it is what gives you autocomplete for `numpy` and `PySide6` and red squiggles on typos.

## Step 6 — Smoke test

Before writing anything, prove the environment works. Create a throwaway script in `scratch/` that:

- imports `numpy`, `scipy.interpolate`, `skimage`, `matplotlib` (with the `QtAgg` backend), and `PySide6.QtWidgets`;
- loads the Shepp–Logan phantom and prints its shape;
- runs one FFT/IFFT round trip and prints the maximum error (expect  $\sim 10^{-13}$ );
- opens a bare Qt window.

If the window opens and the error is tiny, every layer of the stack works.

## 10.3 The project map

```
log_synthesis/
|- .venv/          your private Python -- never committed
|- docs/          the roadmap + this guide (docs/guide/ holds the LaTeX)
|- mri/           THE ENGINE: pure math, no GUI imports, ever
|  |- __init__.py empty; makes 'mri' importable as a package
```

```
| |- kspace_core.py           Milestone 1: FFT <-> image round trip
| |- cartesian_sampling.py    Milestone 2: the four binary masks
| |- filters.py              Milestone 3: apodization windows + noise
| |- trajectories.py          Milestone 4-5: radial/spiral coordinates
| |- gridding.py             Milestone 5: non-Cartesian -> Cartesian grid
| |- metrics.py              MSE / PSNR / SSIM
| |- pipeline.py             the coordinator the GUI calls
|- scratch/                  throwaway experiments and saved plots -- gitignored
|- app.py                    the PySide6 window (Milestone 6). The ONLY file
|                             that imports Qt.
|- requirements.txt          exact package versions -- committed
|- .gitignore                excludes .venv, __pycache__, scratch, LaTeX debris
```

### Why This Matters

**The one architectural rule:** `mri/` is the engine, `app.py` is the face, and the dependency points in exactly one direction — `app.py` imports from `mri`, and nothing inside `mri` ever imports `PySide6` or anything Qt-related. This is what lets you test every milestone from a plain terminal script, lets two people work on engine and GUI simultaneously without merge conflicts, and is the separation Chapter 7 builds its whole architecture on. If you catch yourself typing `from PySide6 import ...` inside the `mri` folder, stop — you are about to break the project.

A few practical conventions that follow from the map:

- **Milestone test scripts** (e.g. `test.week1_math.py`) live at the project *root*, next to `app.py` — then `from mri.kspace_core import ...` works with no path tricks. Run them with `python test.week1_math.py` from the root, venv active.
- `scratch/` is deliberately gitignored: dump experiments, saved figures, and half-ideas there guilt-free. Anything worth keeping graduates into `mri/` or `docs/`.
- **Never commit** `.venv/` or `__pycache__/`; `requirements.txt` is the portable form of the environment.
- **Who edits what:** the file-per-milestone layout means each teammate can own whole files rather than sharing them — the cheapest possible merge-conflict insurance.

### How to Test It Before Moving On

Onboarding is complete when all four are true:

- `python --version` inside the activated venv prints 3.12.x;
- the smoke-test window opens and the round-trip error is  $\sim 10^{-13}$ ;
- VS Code autocompletes `np.fft.` suggestions;
- `git status` does *not* list `.venv/` or `scratch/` as untracked.

# PySide6 Field Guide — Just Enough Qt, in Plain English

## The Concept in Plain English

Chapter 7 designed the application’s architecture; this chapter is the *vocabulary lesson* that lets you actually speak Qt. PySide6 is the official Python binding for **Qt**, the C++ framework behind VLC, OBS Studio, Telegram Desktop, and Maya. You do not need most of Qt — the whole simulator uses perhaps a dozen classes. This chapter introduces exactly those, explains the three ideas that make Qt feel alien at first (the event loop, layouts, and signals), and ends with the styling recipe and the beginner traps.

## 11.1 Idea 1: the event loop — your program becomes a waiter

A terminal script runs top to bottom and exits. A GUI program is different: it starts, then *waits* — for clicks, key presses, slider drags — and reacts to each. That waiting-and-reacting cycle is the **event loop**.

### Analogy

Think of a restaurant waiter. The waiter does not cook anything on their own schedule; they stand ready, and every customer request (an *event*) triggers a short burst of work. If one request takes twenty minutes to handle personally, every other table is ignored — which is exactly why a frozen GUI happens (Chapter 7’s central problem): some handler hogged the waiter.

In practice:

- Every Qt app creates exactly one `QApplication` object at startup — it owns the event loop.
- You build your window, call its `show()` method, then hand control over with `app.exec()`. That call *blocks until the user closes the window*; all your logic from then on runs inside event handlers.
- Corollary: any code you write after `app.exec()` runs only at shutdown. Beginners constantly put setup code there and wonder why it never runs.

## 11.2 Idea 2: widgets and layouts — Lego bricks that resize themselves

Everything visible in Qt is a **widget**, and widgets nest: the window contains panels, panels contain sliders. The dozen widgets this project needs:

| Class       | What it is, in one line                                              |
|-------------|----------------------------------------------------------------------|
| QMainWindow | the outer application window; owns a central widget and a status bar |
| QWidget     | a blank rectangle; the generic container you build panels from       |
| QLabel      | a piece of text (or an image); your metric readouts                  |
| QSlider     | a draggable slider; emits its value as it moves                      |
| QComboBox   | a dropdown; your mask-type and trajectory selectors                  |
| QPushButton | a clickable button; “Load Image”, “Reset”                            |
| QCheckBox   | an on/off toggle; “Apply apodization”                                |
| QGroupBox   | a titled frame that visually groups related controls                 |
| QStatusBar  | the strip along the bottom for transient messages                    |
| QFileDialog | the native “Open File...” dialog, one static call                    |

**Never position widgets with pixel coordinates.** Qt’s answer to arranging things is the **layout manager**: you drop widgets into a layout, and the layout computes positions and recomputes them on every resize.

- `QHBoxLayout` — children in a horizontal row.
- `QVBoxLayout` — children in a vertical column.
- `QGridLayout` — children in rows and columns; your  $2 \times 2$  image-panel grid.
- Layouts nest freely: the main window is one horizontal layout holding [a vertical control column] beside [the grid of four panels] — exactly the sketch in Chapter 7.
- *Stretch factors* decide who gets spare space when the window grows: give the image grid a large stretch and the control column none, and resizing enlarges the images while the sliders stay put.

## 11.3 Idea 3: signals and slots — widgets announce, they never command

The question every beginner asks: “how does my slider *call* my reconstruction function?” Qt’s answer is deliberately indirect.

### Analogy

A widget is like a radio station, not a telephone caller. When a slider moves, it does not reach into your code; it *broadcasts* an announcement — “my value is now 4!” — called a **signal**. Any function may tune in by *connecting* to that signal; a connected function is called a **slot**. One signal may feed many slots, a slot may listen to many signals, and neither side knows the other exists. That ignorance is the feature: your engine code never mentions sliders, so it stays testable without a GUI.

What you actually do:

- Every widget ships with ready-made signals: sliders emit `valueChanged`, buttons emit `clicked`, dropdowns emit `currentIndexChanged`.
- Connecting is one line of the form `widget.signalName.connect(your_function)` — done once, at window construction time.
- Signals pass their payload (the new slider value, the chosen index) as an argument to the slot.

- You can also declare *custom* signals on your own classes — this is precisely how the background worker of Chapter 7 reports “reconstruction finished” back to the window, because signals are the **only** thread-safe bridge in Qt.
- `QSlider` only speaks integers. For a continuous quantity like noise  $\sigma \in [0, 0.2]$ , let the slider run 0..200 and divide by 1000 inside the slot — a standard Qt idiom.

## 11.4 Embedding matplotlib: the canvas is just another widget

The four image panels are matplotlib figures living inside the Qt window. The bridge is one class, `FigureCanvasQTAgg` (from `matplotlib.backends.backend_qtagg`): it *is* a `QWidget`, so it drops into a layout like any label or button. The working rules:

- One **Figure** + one canvas per panel, created **once** at startup. On every update, reuse the existing image object (`set_data`) rather than clearing and redrawing axes — rebuilding axes on every slider tick is the classic cause of a sluggish GUI.
- Request repaints with `draw_idle()`, not `draw()` — it coalesces bursts of updates into one paint, which matters while a slider is being dragged.
- These are images, not plots: switch the axes off, and set the figure face color to match the window background so the panels do not float on white rectangles.
- Import matplotlib *after* selecting the `QtAgg` backend, and never call `plt.show()` inside the app — the Qt window, not pyplot, owns the screen now.

## 11.5 Keeping it responsive: the two-line summary of Chapter 7

The full worker-thread design lives in Chapter 7; the Qt mechanics are:

- Heavy math runs in a worker moved to a `QThread`; results come back via a custom signal carrying the finished arrays.
- **Only the main thread may touch widgets.** The worker computes and emits; the window’s slot repaints. No exceptions.
- Debounce sliders with a single-shot `QTimer` (~150 ms): each movement restarts the timer, and only the timer’s timeout triggers a recomputation — so dragging fires one job, not fifty.

## 11.6 Making it look good: the two-step styling recipe

Out of the box Qt looks dated; two cheap steps fix it.

- **Step 1 — Fusion + palette.** At startup, select the built-in `Fusion` style (identical on every OS, designed to be recolored) and hand `QApplication` a `QPalette` defining your background, surface, text, and one accent color. Twenty lines, transforms the app.
- **Step 2 — Qt Style Sheets (QSS).** A CSS dialect for widgets: you write selectors (`QSlider::handle`, `QGroupBox::title`) and properties for padding, border-radius, and hover states. Use it for rounded control panels, a monospace font on metric readouts, and slider styling. Apply once via `app.setStyleSheet(...)`.
- Remember Chapter 7’s warning: Qt theming does *not* touch the matplotlib canvases — style those from the matplotlib side (figure face color, no axes, a good colormap for the error heatmap), or the app will look like plots bolted into a window.
- Skip QML/Qt Quick entirely for this project — it is a second language to learn and matplotlib embeds poorly in it.

## 11.7 The five classic beginner traps

| Symptom                                                            | Cause and cure                                                                                                                                                            |
|--------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Window flashes open and instantly vanishes                         | You kept no reference to it; Python garbage-collected your window. Store it in a variable that outlives the <code>show()</code> call.                                     |
| GUI freezes during recomputation                                   | Heavy math ran in an event handler on the main thread. Move it to the worker (Chapter 7).                                                                                 |
| Crash: “cannot create children for a parent in a different thread” | A worker touched a widget. Workers emit signals; only the main thread paints.                                                                                             |
| Nothing happens when the slider moves                              | The <code>connect</code> call was never made, or you connected the signal to the <i>result</i> of calling the function (with parentheses) instead of the function itself. |
| Two windows appear, or matplotlib opens its own                    | A stray <code>plt.show()</code> or a second <code>QApplication</code> . One application object, one <code>exec()</code> , no pyplot windows.                              |

### Key Takeaways

- One `QApplication`, one event loop; after `app.exec()` your code lives in slots.
- Widgets go in layouts; layouts nest; pixel positioning is forbidden.
- Widgets broadcast signals; your functions subscribe as slots; the two sides never know each other.
- The matplotlib canvas is a widget: create once, update with `set_data + draw_idle`.
- Compute on the worker thread, paint on the main thread, bridge with signals, debounce with a timer.
- Fusion + palette + a little QSS + styled matplotlib = an app that looks deliberate.

### Self-Check: Can You Answer These?

1. What is the event loop, and why does a long computation inside a slot freeze the entire window?
2. Explain signals and slots to a teammate using the radio-station analogy. Why does this indirection keep the engine GUI-free?
3. Why must widget updates happen only on the main thread, and what is the sanctioned mechanism for a worker to deliver results?
4. Your slider must control  $\sigma \in [0, 0.2]$  but `QSlider` is integer-only — what is the standard idiom?
5. Why is `draw_idle()` preferred over `draw()` when a slider is being dragged?
6. Name the two styling steps that take a default Qt app to a polished-looking one, and the part of the window they *cannot* style.